- Modeling Report: Predicting Customer Satisfaction from Review Text
  - 1. Executive Summary
    - 1.1. Objective and Approach
    - 1.2. Modeling Strategies
    - 1.3. Key Findings & Recommendation
    - **Recommendation**
  - 2. Architectural Overview
    - 2.1. Comparison of Modeling Approaches
    - 2.2. Pipeline Architectures
  - 3. The End-to-End ML Pipeline: A Deeper Look
    - 3.1. Text Preprocessing
    - 3.2. Feature Engineering (`ReviewFeatureExtractor`)
    - 3.3. Hyperparameter Optimization with Optuna
    - 3.4. Experiment Tracking and Reproducibility with MLflow
    - 3.5. Handling Class Imbalance: Oversampling Strategies
    - 3.6. Choosing the Right Evaluation Metric
    - 3.7. Modeling Approach: Transformer-Based Feature Extraction
    - 3.8. Overview of Classifier Models
  - 4. Expected Results & Hypotheses
    - 4.1. Expected Performance Hierarchy
    - 4.2. Expected Classifier Performance
  - 5. Understanding Model Explanations with SHAP
    - 5.1. For Classical (TF-IDF) Models: Standard SHAP
    - 5.2. For Embedding and Transformer Models: Word-Level SHAP
  - 6. Experiment Results and Detailed Analysis
    - 6.1. General Experiment Setup
    - 6.2. Classical Approach Results (TF-IDF)
    - 6.3. Embedding Approach Results
    - 6.4. Transformer Approach Results
    - 6.5. Binary Classification Results (Positive vs. Negative)
    - 6.6. Grand Summary: A “Classical” Victory and Lessons Learned
  - 7. Final Remarks and Outlook
    - 7.1. Final Summary of Findings
    - 7.2. Limitations of this Study
    - 7.3. Future Work & Recommendations
    - 7.4. Final Business Recommendation

Modeling Report: Predicting Customer Satisfaction from Review Text

1. Executive Summary

1.1. Objective and Approach

This project's central objective is to model and predict customer satisfaction from review text. We approached this in two complementary ways to maximize business value:

1. **Primary Objective: Multi-Class Classification (1-5 Stars)** This is our main analytical goal. Predicting the full 1-5 star rating allows for a **nuanced understanding of customer sentiment**. The business value is not just in knowing *if* a customer is unhappy, but *why* they are only "neutral" (3-stars) versus "dissatisfied" (2-stars). This model serves as a deep-dive analytical tool to identify the specific drivers of "lukewarm" reviews, which are often the most actionable.

2. **Secondary Objective: Binary Classification (Positive/Negative)** Recognizing that the multi-class task is challenging and that the business also has a high-priority need for a simple, high-accuracy flag, we formulated a secondary task. By mapping 1/2-star reviews to "Negative" and 4/5-star reviews to "Positive" (dropping 3-star reviews), we aimed to build a model for **proactive operational monitoring**. The value here is speed and reliability, enabling automated alerts for rapid customer service intervention.

1.2. Modeling Strategies

To find the best solution, we explored three distinct modeling paradigms, each testing a different method of text vectorization:

1. **Classical Approach (TF-IDF):** A robust baseline using word frequencies (TF-IDF), which excels at keyword identification.
2. **Embedding Approach (Word2Vec, FastText):** A method to capture semantic meaning by representing words as dense vectors.

3. **Transformer Approach (BERT-family):** A state-of-the-art approach using pre-trained models to generate deep, context-aware embeddings.

1.3. Key Findings & Recommendation

Our experiments yielded a clear and consistent set of findings that defied our initial hypotheses. The `Classical (TF-IDF)` approach, combined with a `LogisticRegression` model, proved to be the most robust and high-performing solution for **both** tasks.

- **Finding 1 (Approach):** The `Classical (TF-IDF)` strategy **overwhelmingly outperformed** all `Embedding` strategies for both multi-class and binary problems. The hypothesis that semantic `Embedding` models would be superior (Section 4.1) was proven false.
- **Finding 2 (Multi-Class Result):** For our primary 1-5 star objective, the **`LogisticRegression (TF-IDF)`** model was the best, achieving a **Weighted F1-score of 0.7347**. Its analysis (Section 6.2.2) shows it excels at identifying the 1-star and 5-star extremes but, as expected, struggles with the nuanced 2, 3, and 4-star classes.
- **Finding 3 (Binary Result):** For the secondary positive/negative task, the **`LogisticRegression (TF-IDF)`** model was the clear winner, achieving an excellent **Weighted F1-score of 0.9392**. Crucially, it showed minimal overfitting and a high recall (92.1%) for the minority "negative" class, making it highly reliable.
- **Finding 4 (Transformer):** The Transformer experiments were not completed. As detailed in Section 6.4, this was due to a computational bottleneck in our initial "on-the-fly" embedding generation, not a failure of the model itself.

Recommendation

We propose a **dual-model recommendation** to maximize business value:

1. **For Production & Alerting: Use the Binary `LogisticRegression (TF-IDF)` Model (F1: 0.939)** This model is fast, robust, and highly accurate. It should be deployed for all operational tasks requiring a high-confidence "Positive" or "Negative" flag, such as real-time customer service alerts.

2. **For Analytics & Insight: Use the Multi-Class `LogisticRegression (TF-IDF)` Model (F1: 0.735)** While its overall F1 is lower, this model is our only

tool for understanding the “why” behind neutral and dissatisfied reviews. It should be used by the analytics team to perform SHAP-based deep dives on the drivers of 2, 3, and 4-star reviews.

---

2. Architectural Overview

2.1. Comparison of Modeling Approaches

The following table summarizes the core characteristics and trade-offs of the three modeling strategies implemented.

Feature	Classical (TF-IDF)	Embedding (Word2Vec, etc.)	Transformer (BERT)
Core Idea	Counts weighted word frequencies.	Represents words as vectors in a semantic space.	Understands text contextually, like a human.
Vector Type	Sparse (many zeros)	Dense	Dense
Dimensionality	High (10,000 - 50,000+)	Low (e.g., 300)	Medium (e.g., 768)
Handles OOV Words?	No	Only FastText	Yes
Captures Semantics?	No (treats “good”/“great” as different)	Yes (knows “good”/“great” are similar)	Yes, deeply
Context-Aware?	No	No (vector for “bank” is always the same)	**Yes** (vector for “bank” differs in context)

Feature	Classical (TF-IDF)	Embedding (Word2Vec, etc.)	Transformer (BERT)
Interpretability	High (feature names are words)	Low (features are abstract dimensions)	Very Low (requires word-level SHAP)
Computational Cost	Low	Medium	High

2.2. Pipeline Architectures

This section details the specific architecture of each of our three experimental pipelines. A key design principle is that **all models receive a combined feature set**: one part from the primary text vectorization method, and another from a common set of engineered numerical features.

2.2.1. Architectural Commonalities and Differences

While the three pipelines test fundamentally different NLP techniques, they share a common, modular structure designed for fair comparison.

Similarities: - **Parallel Feature Generation:** All three pipelines use a `ColumnTransformer` to process the raw text in two parallel paths: one for high-dimensional text vectorization and one for low-dimensional numerical feature engineering. - **Unified Feature Set:** The outputs of both paths are always concatenated into a single feature matrix. This ensures that every classifier, regardless of the pipeline, benefits from both the vectorized text and the engineered features. - **Consistent Preprocessing:** The same initial text cleaning and normalization steps are applied to the text before it enters either the vectorization or feature engineering path, ensuring a consistent starting point.

Key Difference: The core distinction lies in **Path 1 (Text Vectorization)**. Each pipeline substitutes a different vectorization engine: 1. **Classical:** `TfidfVectorizer` creates a wide, sparse matrix based on word counts. 2. **Embedding:** A custom `EmbeddingVectorizer` (using FastText) creates a short, dense vector representing semantic meaning. 3. **Transformer:** A `TransformerFeatureExtractor` (using BERT) creates a medium-length, dense vector representing deep contextual understanding.

This single change has a cascading effect, influencing the nature of the feature space (sparse vs. dense), the choice of optimal classifier (linear vs. tree-based), and the method required for model interpretation (standard SHAP vs. word-level SHAP).

2.2.2. Classical Pipeline Architecture

The classical approach uses TF-IDF, a method that excels at identifying important keywords. The pipeline is structured to create a very wide, sparse feature matrix representing word frequencies, which is then concatenated with the handful of numerical features before being fed to a classifier that is well-suited for this type of data, such as `LogisticRegression`.

Detailed Diagram:

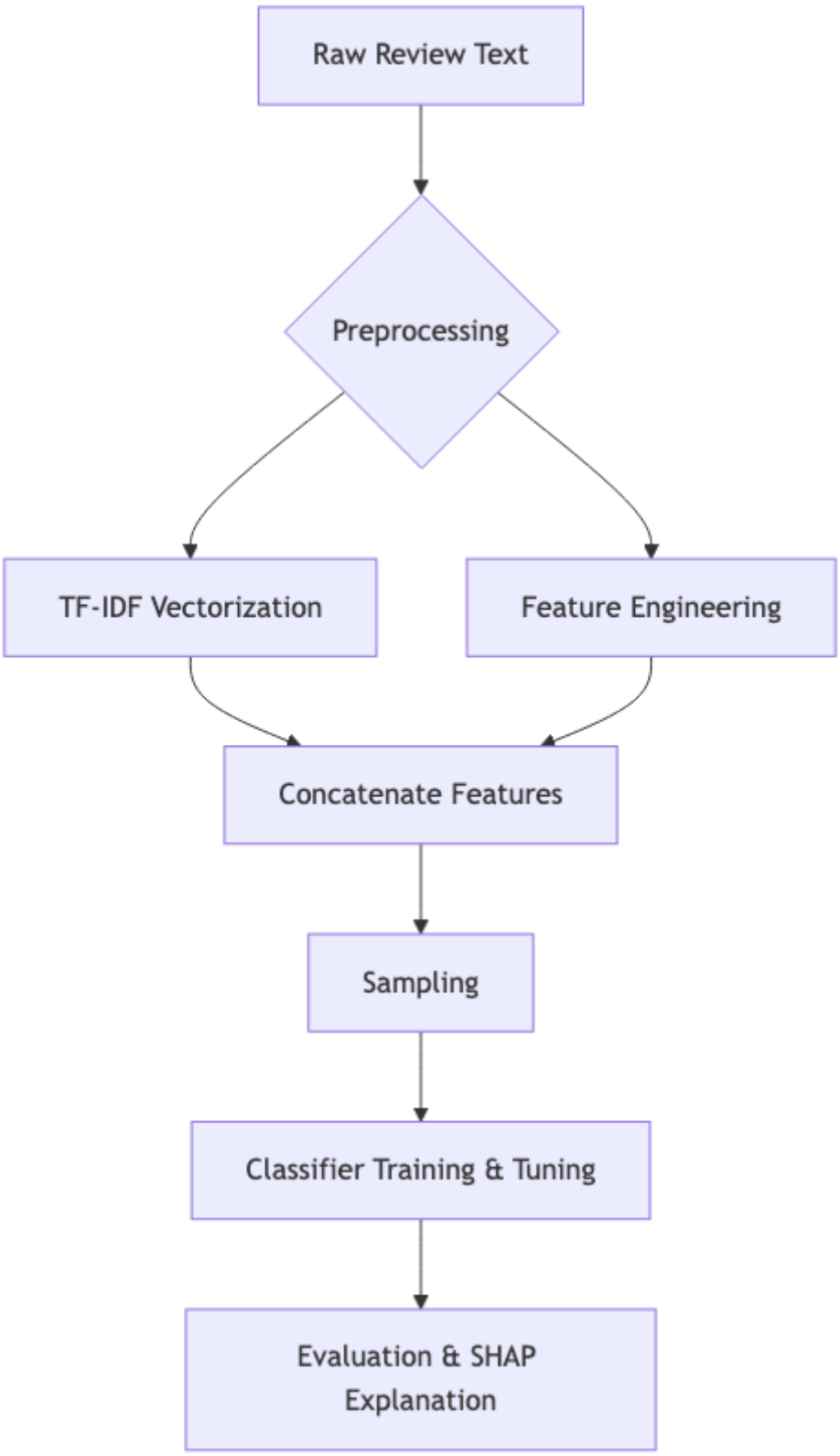
```
=========================================================
==========================================
|                                   Classical Model
Pipeline                                        |
=========================================================
==========================================
  Input: DataFrame (containing 'text' column)

    |
    ▼
┌──────────────────────────────────────────────────────────
──────────────────────────────────────────┐
| ColumnTransformer: Applies different transformations
to the text data in parallel.             |
|
                                          |
|   ├─ Path 1: Text Vectorization
──────────────────────────────────────────────────────────
───────┤
|   |   Input: 'text'
column
                  |
|   |
|
                                |
|   |
```

```
▼
                                  │
│   │   ┌──────────────────────────┐
┌──────────────────────────────────────────────────────────┐   │
│   │   │  Step 1: Preprocessing   │──►  │         Step
2: TF-IDF Vectorization             │   │
│   │   │ (e.g., lowercase, etc.)  │      │ (Creates a
~10,000+ dimension sparse vector)     │   │
│   │   └──────────────────────────┘
└──────────────────────────────────────────────────────────┘   │
│
│
                                          │
│   ├─ Path 2: Numerical Feature Engineering
──────────────────────────────────────────────────────────────┤
│   │   Input: 'text'
column
                   │
│   │
│
                                  │
│   │
▼
                                  │
│   │
┌──────────────────────────────┐
                                    │
│   │   │  ReviewFeatureExtractor
│
     │
│   │   │ (e.g., word_count, etc.)
│
     │
│   │
└──────────────────────────────┘
                                    │
│
                                              │
└──────────────────────────────────────────────┬──────────────────────────────────────────────┘
                                               │
```

```
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ Feature Union: Concatenates the sparse TF-IDF vectors and the dense numerical features.                │
│   Output: A single, very wide feature matrix.                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ Sampling: (e.g., RandomOverSampler)                                                                    │
│   Balances class distribution in training data.                                                        │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ Classifier: e.g., LogisticRegression, trained on balanced data.                                        │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ▼
                                    Output:
                            Predicted Star Rating (1-5)
```

Summary Diagram:

> *How to read this diagram: Each `[Node]` represents a data processing step. The `-->` arrows indicate the flow of data through the pipeline from the raw text input to the final prediction.*



2.2.3. Embedding Pipeline Architecture

This approach replaces the sparse TF-IDF vector with a dense vector that captures semantic meaning. We use pre-trained models like Word2Vec, GloVe, and FastText. The key step is aggregating the individual word vectors into a single

vector for the entire review, for which we use advanced methods like TF-IDF weighted averaging or FastText's superior `get_sentence_vector` function.

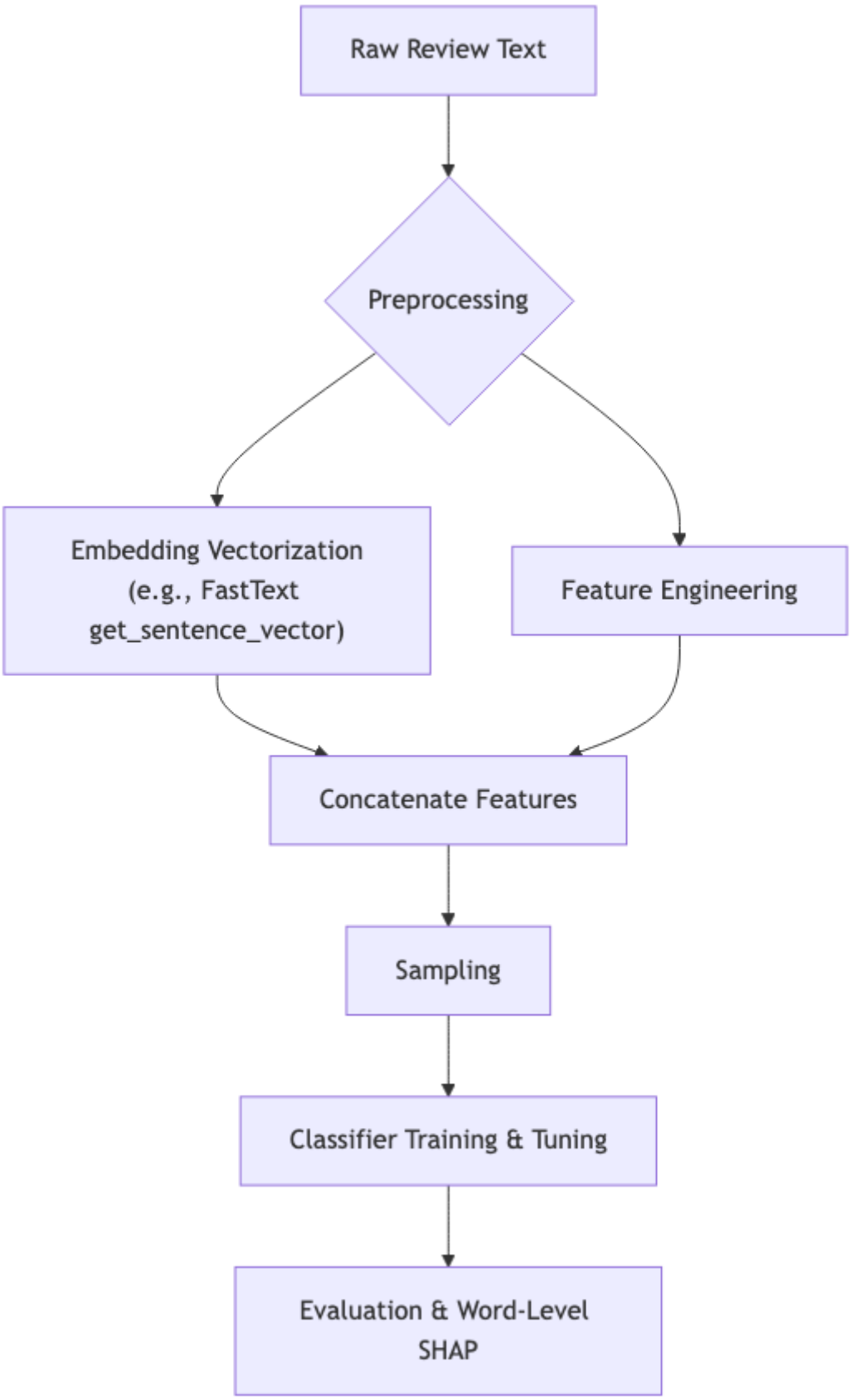
Detailed Diagram:

```
============================================================
==========================================
|                                   Embedding Model
Pipeline                                          |
============================================================
==========================================
  Input: DataFrame (containing 'text' column)

    |
    ▼
┌──────────────────────────────────────────────────────────
──────────────────────────────────────────┐
│ ColumnTransformer: Applies different transformations
to the text data in parallel.             |
|
                                          |
|   ├─ Path 1: Text Vectorization
──────────────────────────────────────────────────────────
───────┤
|   |   Input: 'text'
column
                  |
|   |
|
                                |
|   |
▼
                                |
|   |   ┌──────────────────────────┐
┌──────────────────────────────────────────────────────┐   |
|   |   |  Step 1: Preprocessing   |──►  |      Step 2:
Embedding Vectorization             |   |
|   |   | (e.g., lowercase, etc.)  |      | (e.g.,
FastText creating a 300-dim dense vector) |   |
|   |   └──────────────────────────┘
└──────────────────────────────────────────────────────┘   |
```

```
│
│
                                        │
│   ├─ Path 2: Numerical Feature Engineering
──────────────────────────────────────────────────────────┤
│   │   Input: 'text'
column
                  │
│   │
│
                                │
│   │
▼
                                │
│   │
┌──────────────────────────┐
                                  │
│   │   │  ReviewFeatureExtractor
│
     │
│   │   │ (e.g., word_count, etc.)
│
     │
│   │
└──────────────────────────┘
                                  │
│
                                          │
└──────────────────────────────────────────┴──────────────────────────
──────────────────────────────────────────┘
                                          │
                                          ▼
┌──────────────────────────────────────────────────────────────────────
──────────────────────────────────────────┐
│ Feature Union: Concatenates the dense embedding vector
and the dense numerical features.       │
│   Output: A single dense feature matrix (e.g., 300 +
10 = 310 features).                       │
└──────────────────────────────────────────┴──────────────────────────
──────────────────────────────────────────┘
                                          │
```

```
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│ Sampling: (e.g., RandomOverSampler)                                                      │
│   Balances class distribution in training data.                                          │
└──────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ▼
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│ Classifier: e.g., GradientBoostingClassifier, trained on balanced data.                  │
└──────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ▼
                                   Output:
                           Predicted Star Rating (1-5)
```

Summary Diagram:


Raw Review Text
Preprocessing
Embedding Vectorization (e.g., FastText get_sentence_vector)
Feature Engineering
Concatenate Features
Sampling
Classifier Training & Tuning
Evaluation & Word-Level SHAP

2.2.4. Transformer Pipeline Architecture

This state-of-the-art approach uses a large, pre-trained transformer model as a feature extractor. It processes the entire raw text and outputs a single, contextually rich vector that understands nuances, sarcasm, and complex sentence structures. This powerful vector is then combined with our engineered features to be used by the final classifier.

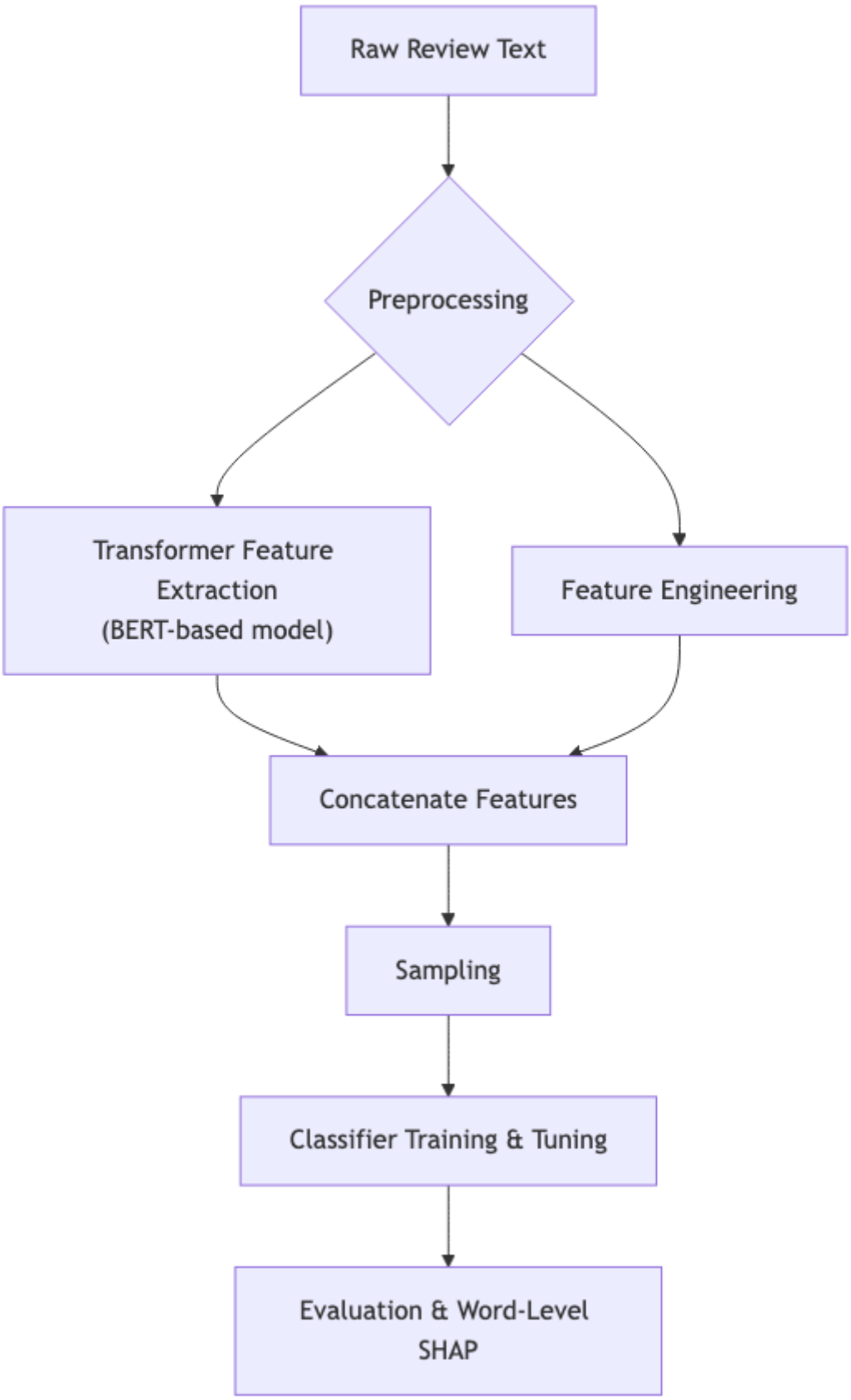
Detailed Diagram:

```
============================================================
==========================================
|                                  Transformer Model
Pipeline                                     |
============================================================
==========================================
  Input: DataFrame (containing 'text' column)

    |
    ▼
┌──────────────────────────────────────────────────────────
──────────────────────────────────────────┐
│ ColumnTransformer: Applies different transformations
to the text data in parallel.             |
|
                                          |
|   ├─ Path 1: Text Vectorization
──────────────────────────────────────────────────────────
───────┤
|   |   Input: 'text'
column
                  |
|   |
|
                                |
|   |
▼
                                |
|   |   ┌──────────────────────────┐
┌──────────────────────────────────────────────┐   |
|   |   |  Step 1: Preprocessing   |──►  |   Step 2:
```

```
Transformer Feature Extraction         │   │
│   │   │ (e.g., lowercase, etc.)  │      │ (e.g., BERT
creating a 768-dim dense vector)     │   │
│   │   └──────────────────────────┘
└──────────────────────────────────────────────────────┘   │
│
│
                                        │
│   ├─ Path 2: Numerical Feature Engineering
──────────────────────────────────────────────────────────┤
│   │   Input: 'text'
column
                   │
│   │
│
                                │
│   │
▼
                                │
│   │
┌──────────────────────────┐
                                  │
│   │   │  ReviewFeatureExtractor
│
     │
│   │   │ (e.g., word_count, etc.)
│
     │
│   │
└──────────────────────────┘
                                  │
│
                                          │
└──────────────────────────────────────────┴──────────────────────
──────────────────────────────────────────┘
                                          │
                                          ▼
┌──────────────────────────────────────────────────────────────
──────────────────────────────────────────────┐
│ Feature Union: Concatenates the dense transformer
vector and the dense numerical features.     │
```

```
│   Output: A single dense feature matrix (e.g., 768 +
10 = 778 features).                          │
└──────────────────────────────────────────┬──────────────────────────
──────────────────────────────────────────┘
                                           │
                                           ▼
┌──────────────────────────────────────────────────────────────────────
──────────────────────────────────────────┐
│ Sampling: (e.g.,
RandomOverSampler)
                       │
│   Balances class distribution in training
data.                                                          │
└──────────────────────────────────────────┬──────────────────────────
──────────────────────────────────────────┘
                                           │
                                           ▼
┌──────────────────────────────────────────────────────────────────────
──────────────────────────────────────────┐
│ Classifier: e.g., GradientBoostingClassifier, trained
on balanced data.                            │
└──────────────────────────────────────────┬──────────────────────────
──────────────────────────────────────────┘
                                           │
                                           ▼
                                       Output:
                              Predicted Star Rating (1-5)
```

Summary Diagram:


Raw Review Text
Preprocessing
Transformer Feature Extraction (BERT-based model)
Feature Engineering
Concatenate Features
Sampling
Classifier Training & Tuning
Evaluation & Word-Level SHAP

3. The End-to-End ML Pipeline: A Deeper Look

3.1. Text Preprocessing

The `PreprocessingPipeline` in `src/preprocessing/preprocessing_pipeline.py` is applied consistently across all experiments. It performs several crucial steps to clean and standardize the review text while preserving important sentiment signals:

The order of these operations is critical for correctness and effectiveness: 1. **Lowercase Conversion:** This is the first step. It unifies the text (e.g., 'Good', 'good', 'GOOD' all become 'good'), dramatically reducing the vocabulary size and preventing the model from treating the same word differently based on capitalization. 2. **URL and HTML Removal:** Performed early to remove non-linguistic noise that could interfere with subsequent steps like contraction expansion. 3. **Emoji & Emoticon Conversion:** Emojis are converted to descriptive text. This is a vital step that transforms pictorial sentiment signals into textual features the model can easily learn from. It's done before punctuation removal to protect the descriptive text (e.g., `\_smiling\_face\_`). 4. **Contraction Expansion:** `don't` is expanded to `do not`. This is crucial for accurate tokenization and negation handling. It must be done before punctuation removal, as the apostrophe in `don't` is the key to identifying it as a contraction. 5. **Negation Handling:** Words following a negation (like 'not', 'no', 'never') are appended with a `\_NEG` suffix (e.g., `not good` -> `not good\_NEG`). This teaches the model that `good\_NEG` is the opposite of `good`, solving a classic problem where bag-of-words models struggle with negation. 6. **Punctuation Removal and Whitespace Standardization:** These final cleanup steps remove any remaining special characters and ensure consistent spacing, preparing the text for clean tokenization by the vectorizer.

3.2. Feature Engineering (`ReviewFeatureExtractor`)

In parallel with vectorization, we extract a set of numerical features from the raw text using the `ReviewFeatureExtractor` defined in `src/preprocessing/feature_engineering.py`. These features provide the classifier with additional, complementary information about the review's style and structure.

The final feature set passed to the classifier is a combination of the high-dimensional vectorized text and these low-dimensional engineered features.

Based on the exploratory data analysis, these features were selected because they showed potential correlations with the target variable (star rating) and capture stylistic nuances that pure text vectorization might miss.

Feature	Description	Why it's useful for sentiment analysis (Insights from EDA)
`word_count`	Total number of words in the review.	Very short reviews are often 5-star ("Great!"), while extremely long reviews tend to be 1-star, detailing a series of problems. This feature helps capture that non-linear relationship.
`char_count`	Total number of characters.	A similar, but slightly different, measure of review length and effort.
`exclamation_count`	Number of exclamation marks (`!`).	A high count is a strong indicator of intense emotion. EDA shows it's common in both 1-star ("This is a disaster!!!") and 5-star ("Love it!!!") reviews, making it a powerful feature for identifying polarized opinions.
`question_count`	Number of question marks (`?`).	Frequently appears in negative reviews where customers are expressing confusion, seeking help, or questioning service quality ("Why hasn't my package arrived?").
`all_caps`	Number of words written in ALL CAPS.	A very strong signal of emphasis or "shouting." EDA confirms this is far more prevalent in highly negative reviews.
		Normalizes the `all_caps` count. A review with 3 all-caps

Feature	Description	Why it's useful for sentiment analysis (Insights from EDA)
`capital_ratio`	Ratio of all-caps words to total words.	words out of 10 is more "shouty" than one with 3 out of 100. This provides better context than the raw count.
`punctuation_ratio`	Ratio of punctuation characters to total characters.	Can indicate a more emotional or less formal writing style. A high ratio, especially with exclamation marks, points to a more passionate review.
`avg_word_len`	Average length of words in the review.	Can be a proxy for vocabulary complexity. EDA might show that more descriptive, negative reviews use longer, more specific words.
`sentence_count`	Total number of sentences (requires spaCy).	Measures the structural complexity. A single-sentence review is stylistically different from a multi-paragraph one.
`unique_ratio`	Ratio of unique words to total words.	Measures lexical diversity. A low ratio might indicate repetition, often seen in frustrated rants ("late late late"). A high ratio could suggest a more thoughtful, detailed review.

3.3. Hyperparameter Optimization with Optuna

We use **Optuna** for systematic and efficient hyperparameter tuning for every classifier in every experiment. Optuna is an open-source hyperparameter optimization framework that automates the search for optimal hyperparameter values for machine learning models.

How Optuna Works: Unlike traditional methods, Optuna employs a dynamic, “define-by-run” API, allowing users to construct the search space on the fly. Its core strength lies in its intelligent sampling and pruning strategies: - **Sampling Algorithms:** Optuna uses advanced sampling algorithms (e.g., Tree-structured Parzen Estimator (TPE), CMA-ES) to intelligently propose new hyperparameter combinations based on the performance of previous trials. This is more efficient than random or grid-based approaches. - **Pruning Mechanisms:** It can automatically stop unpromising trials early in the training process. For example, if a model’s performance on a validation set is significantly worse than other trials after only a few epochs, Optuna can “prune” that trial, saving computational resources.

The `run_optuna_for_model` function in our pipeline optimizes for the **weighted F1-score** using cross-validation. This ensures that we find the best possible configuration for each model automatically, considering both precision and recall across all classes, which is crucial for imbalanced datasets. All study results, trials, and parameter importances are logged to MLflow for complete transparency and reproducibility.

Comparison to Conventional Approaches:

Feature	**Optuna**	**Scikit-learn Grid Search**	**Scikit-learn Random Search**	**Scikit-learn Bayes Search (e.g., `BayesSearchCV`)**
Search Strategy	Intelligent (TPE, CMA-ES, etc.)	Exhaustive (all combinations)	Random sampling	Probabilistic (Gaussian Process, etc.)
Efficiency	High (focuses on promising regions, prunes)	Low (explores entire space)	Medium (better than grid for large spaces)	High (learns from past trials)
Pruning	Yes (stops unpromising trials early)	No	No	No (though some implementations might add it)

Feature	Optuna	Scikit-learn Grid Search	Scikit-learn Random Search	Scikit-learn Bayes Search (e.g., `BayesSearchCV`)
Search Space	Dynamic, define-by-run	Fixed, pre-defined	Fixed, pre-defined	Fixed, pre-defined
Parallelization	Yes, distributed	Yes (embarrassingly parallel)	Yes (embarrassingly parallel)	More complex, often sequential for Bayesian update
Best Use Case	Complex, large search spaces, resource-limited	Small, well-understood search spaces	Large search spaces, quick exploration	Complex, expensive evaluations, learns quickly

Why Optuna is Suitable/Preferable for Our Task: 1. **Efficiency with Complex Models:** Given that we are exploring three distinct modeling paradigms (Classical, Embedding, Transformer), each with potentially many hyperparameters, Optuna's intelligent sampling and pruning significantly reduce the computational time and resources required compared to Grid Search or even Random Search. 2. **Dynamic Search Space:** The "define-by-run" API allows for flexible definition of hyperparameter ranges, which can be particularly useful when experimenting with different vectorizers or classifiers that might have unique parameter sets. 3. **Robustness:** By optimizing for weighted F1-score, Optuna helps us find models that perform well across all star ratings, addressing potential class imbalance issues inherent in customer review datasets. 4. **Scalability:** Optuna is designed for distributed optimization, making it suitable for larger-scale experiments or when more computational power becomes available. 5. **Transparency:** Its integration with MLflow ensures that all optimization trials, their parameters, and results are meticulously logged, providing a clear audit trail and facilitating analysis of hyperparameter importance.

In summary, Optuna provides a powerful and flexible framework that allows us to efficiently navigate complex hyperparameter landscapes, leading to better-performing models while managing computational costs effectively.

3.4. Experiment Tracking and Reproducibility with MLflow

To ensure every experiment is logged, organized, and fully reproducible, we use **MLflow Tracking**. MLflow is an open-source platform for managing the end-to-end machine learning lifecycle. For this project, we leverage its tracking capabilities as a centralized repository for all our modeling efforts.

How We Utilize MLflow:

Every single model training execution—including each individual trial within an Optuna hyperparameter search—is automatically logged as a distinct **run** in MLflow. For each run, we meticulously record:

- **Parameters:** All hyperparameters used for the model, vectorizer, and feature engineering steps. This provides a complete configuration snapshot.
- **Metrics:** Key performance indicators such as weighted F1-score, accuracy, precision, and recall for both the training and test sets. This allows for direct, quantitative comparison between models.
- **Artifacts:** Any file associated with the run is stored as an artifact. This is the most powerful feature for our purposes, and we log:
  - The trained model object itself, packaged in a standard format (`mlflow.sklearn.log_model`).
  - Evaluation plots, including confusion matrices and Optuna hyperparameter importance charts.
  - All generated SHAP explanation plots (bar, beeswarm, and word-level text plots).
  - The recreation data needed to regenerate SHAP plots on demand, as handled by our `practical_enhanced_storage.py` utility.
- **Source Code & Tags:** MLflow automatically records the Git commit hash, ensuring we can always trace a result back to the exact version of the code that produced it. We also use tags to categorize runs by approach (e.g., `classical`, `embedding`).

Benefits for Our Project:

1. **Centralized & Searchable Results:** The MLflow UI provides a single, web-based dashboard to view, sort, filter, and compare all experiments, eliminating the need for manual spreadsheets.
2. **Guaranteed Reproducibility:** With parameters, code versions, and data artifacts logged, we can confidently and accurately reproduce any result from any point in the project's history.

3. **Effortless Model Comparison:** We can easily identify the best-performing model across all three approaches by simply sorting the runs by our target metric (e.g., `Test F1 Weighted`).
4. **Simplified Model Management:** Logged models are automatically versioned and can be managed through the MLflow Model Registry, streamlining the path from experimentation to deployment.

3.5. Handling Class Imbalance: Oversampling Strategies

As highlighted in the Exploratory Data Analysis (EDA), our dataset exhibits significant class imbalance, with a disproportionately high number of 5-star reviews compared to 1-star or 2-star reviews. This is a common challenge in real-world classification tasks and can lead to models that are biased towards the majority class, performing poorly on the critical minority classes.

The Problem with Imbalance: When a model is trained on imbalanced data, it tends to optimize for overall accuracy, which can be misleading. A model might achieve high accuracy by simply predicting the majority class most of the time, effectively ignoring the minority classes. For customer satisfaction, accurately identifying negative reviews (1-star, 2-star) is often more critical for business intervention than correctly classifying every 5-star review.

Strategies to Address Class Imbalance: Several techniques exist to mitigate the effects of class imbalance:

1. **Oversampling:** Increases the number of samples in the minority class(es).
    - **Pros:** No loss of information from the original dataset. Can help the model learn more robust decision boundaries for minority classes.
    - **Cons:** Can lead to overfitting on the minority class if not used carefully. Increases training time and memory usage.
    - **Examples:**
        - **RandomOverSampler:** Duplicates random samples from the minority class. Simple and effective.
        - **SMOTE (Synthetic Minority Over-sampling Technique):** Generates synthetic samples for the minority class by interpolating between existing minority class instances. This creates new, distinct samples rather than just duplicating existing ones, reducing the risk of overfitting.
2. **Undersampling:** Decreases the number of samples in the majority class(es).
    - **Pros:** Reduces training time and memory requirements.

    - **Cons:** Can lead to a loss of potentially valuable information from the majority class, which might be important for overall decision-making.
    - **Examples:** Random undersampling, NearMiss, Tomek Links.
3. **Cost-Sensitive Learning:** Modifies the learning algorithm to penalize misclassifications of minority classes more heavily than majority classes.
    - **Pros:** Directly addresses the business cost of misclassification. Does not alter the dataset distribution.
    - **Cons:** Requires careful tuning of cost parameters, which can be complex.
4. **Ensemble Methods:** Techniques like Bagging or Boosting can be adapted for imbalanced datasets (e.g., EasyEnsemble, BalanceCascade).

Our Approach: Leveraging Oversampling

In this project, we primarily utilize **oversampling strategies** to address the class imbalance. Our implementation, found in `src/utils/sampling_utils.py`, integrates techniques such as `RandomOverSampler` and `SMOTE` from the `imbalanced-learn` library.

Why Oversampling? We chose oversampling as our primary strategy to ensure that our models have sufficient data to learn the characteristics of the underrepresented (and often more critical) 1-star and 2-star reviews. This approach avoids discarding any original data, which is crucial when dealing with potentially limited datasets, and aims to improve the model's ability to generalize to these minority classes.

A Note on SMOTE with Textual Data: While SMOTE is a powerful technique, its application to textual data requires careful consideration. SMOTE generates synthetic data by creating "averages" of existing data points in the feature space. - **With TF-IDF:** The high-dimensional and sparse nature of TF-IDF vectors means that interpolating between two vectors can create a new, dense vector with many small, non-integer values. This synthetic vector does not resemble a real document's TF-IDF representation, potentially introducing noise rather than a useful signal. - **With Embeddings:** While less problematic than with TF-IDF, creating a synthetic sentence embedding by averaging two existing ones does not guarantee that the new vector corresponds to a semantically coherent sentence.

Because of these challenges, simpler methods like `RandomOverSampler` are often a more robust and safer starting point for text data, as they do not create artificial feature combinations. Our pipeline is configured to test both, allowing us to empirically determine the most effective strategy.

Integration into the Pipeline: The chosen oversampling technique is incorporated as a dedicated step within our scikit-learn pipelines, positioned *after* feature extraction but *before* the final classifier. This ensures that: 1. Oversampling is applied **only to the training data** during cross-validation, preventing data leakage from the test set. 2. The feature space is already defined (e.g., as TF-IDF vectors or dense embeddings) before the sampling is applied. This ensures that oversampling techniques operate on meaningful numerical representations of the text, rather than on the raw text itself.

By systematically applying oversampling, we aim to build models that are not only accurate but also fair in their predictive performance across all customer satisfaction levels, providing more actionable insights for business stakeholders.

3.6. Choosing the Right Evaluation Metric

Given the class imbalance in our dataset, relying on accuracy alone is insufficient and can be highly misleading. A model could achieve high accuracy by simply predicting the majority class (e.g., 5-star reviews) most of the time, while failing to identify the more critical minority classes (e.g., 1-star reviews). Therefore, we must use metrics that provide a more nuanced view of performance.

For this project, we optimize our models using the **weighted F1-score**.

Why Weighted F1-Score? The F1-score is the harmonic mean of precision and recall, providing a single score that balances both concerns. The “weighted” variant calculates the F1-score for each class independently and then computes a weighted average based on the number of true instances for each class (its support). This makes it an excellent choice for multi-class, imbalanced problems because: - It considers both false positives (precision) and false negatives (recall). - It accounts for the class imbalance, giving more importance to the performance on larger classes but without completely ignoring the smaller ones. - It provides a single, comprehensive metric that is ideal for automated optimization processes like our Optuna hyperparameter search.

Common Classification Metrics:

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
Accuracy	The percentage	$\frac{TP + TN}{TP +	**Use Case:** A good	**Pitfall:** Highly

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
	of all predictions that were correct.	TN + FP + FN}$	starting metric when the dataset classes are **well-balanced**. **Interpretation:** “Out of all predictions, what fraction did the model get right?”	misleading on **imbalanced datasets**. A model that predicts the majority class 99% of the time will have 99% accuracy, even if it completely fails to identify the rare minority class.
Precision	Of all the times the model predicted **positive**, what percentage was correct?	$\frac{TP}{TP + FP}$	**Use Case:** When the cost of a **False Positive (FP)** is high. **Examples:** • Email spam detection (you don’t want important emails in spam). • Recommending expensive products.	**Caveat:** Precision alone says nothing about the False Negatives. A model that only predicts one positive class (and gets it right) has 100% precision but is useless if it misses all other positives.

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
			Interpretation: “How trustworthy is a positive prediction from my model?”	
Recall (Sensitivity)	Of all the **actual positive cases**, what percentage did the model correctly identify?	$\frac{TP}{TP + FN}$	**Use Case:** When the cost of a **False Negative (FN)** is high. **Examples:** • Medical diagnosis for a serious disease. • Fraud detection (you don’t want to miss a fraudulent transaction). **Interpretation:** “How well did the model find all the positive cases?”	**Caveat:** Recall alone says nothing about False Positives. A model that predicts “positive” for every single sample will have 100% recall but will be useless due to massive numbers of false positives.
F1-Score	The **harmonic mean** of Precision and Recall.	$2 \times \frac{\text{Precision} \times	**Use Case:** The best “all-in-one” metric for **im**	**Caveat:** It’s an abstract metric and harder to

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
	It balances both metrics.	\text{Recall}}{\text{Precision} + \text{Recall}} $	**balanced datasets** when you care about both FPs and FNs. **Interpretation:** A single score that represents a model's performance on the positive class. A high F1-score means the model has low FPs and low FNs.	explain to non-technical stakeholders than accuracy. It also ignores the True Negatives.
Specificity (TNR)	Of all the **actual negative cases**, what percentage did the model correctly identify?	$\frac{TN}{TN + FP}$	**Use Case:** When correctly identifying negatives is critical. It's the "Recall for the negative class." **Examples:** • A legal system ("innocent until proven guilty").	**Caveat:** Rarely used as the *primary* metric, but it's very useful for understanding the full picture alongside Recall (Sensitivity).

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
			• Drug testing (you don't want to falsely accuse someone).	
ROC AUC (Area Under Curve)	Measures the model's ability to **distinguish between classes** across all possible thresholds.	(Area under the ROC curve)	**Use Case:** Evaluating a model's *overall discriminative power*, independent of a specific classification threshold. Excellent for comparing models. **Interpretation:** • **1.0:** Perfect classifier. • **0.5:** Useless (random guessing). • **< 0.5:** Worse than random.	**Pitfall:** Can be overly optimistic on **highly imbalanced** datasets, as it includes TNs in its calculation. The score itself isn't intuitive (e.g., "What does 0.82 AUC mean?").
Log Loss (Cross-Entropy)	Measures the performance of a model	$- \frac{1}{N} \sum_{i=1}^{N} [y_i	**Use Case:** When you care about the *confiden*	**Caveat:** The score is sensitive to outliers and

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
	that outputs **probabilities** rather than hard labels.	\log(p_i) + (1-y_i) \log(1-p_i)]$	*ce* of the predictions. It heavily penalizes overconfident wrong answers. **Interpretation:** Lower is better. A perfect model has a log loss of 0.	is not easily interpretable. It's best used for comparing models, not for an absolute measure of "goodness."
Weighted F1	An F1-score averaged across all classes, **weighted by the number of samples** in each class (support).	(F1-score for class 1 × % of class 1) + … + (F1-score for class N × % of class N)	**Use Case:** Your project (multi-class rating prediction). It's the best single metric for **imbalanced multi-class problems** as it fairly represents the model's performance on all classes.	**Caveat:** The score can be inflated by high performance on the majority class. If minority class performance is critical, also check the **Macro F1**.
Macro F1	An F1-score averaged across all classes, where **each**	$ \frac{\text{F1-score for class 1} + ... + \text{F1-	**Use Case:** Multi-class classification where you care about	**Caveat:** Poor performance on a rare class will

Metric	Description	Formula	Use Case & Interpretation	Caveats & Pitfalls
	class is treated equally (unweighted).	score for class N}}{N} $	performance on **all classes equally**, regardless of how rare they are. **Interpretation:** A good score here means the model performs well even on rare classes.	heavily penalize the overall score, which may not be desirable if that class isn't important.
Confusion Matrix	A table visualizing the model's performance. It is the **source** for calculating most other metrics.	`[[TN, FP], [FN, TP]]`	**Use Case:** The single best tool for **diagnosing** a model. It shows you *exactly* where the model is getting confused. **Interpretation:** "Which classes are being mistaken for which other classes?"	**Caveat:** Not a single-number metric. It's a diagnostic tool, not a final score for comparison. It can become very large and hard to read in multi-class problems.

3.7. Modeling Approach: Transformer-Based Feature Extraction

To move beyond static, non-contextual embeddings (like Word2Vec), we explored an approach using Transformer models. This family of models represents the current state-of-the-art in Natural Language Processing (NLP) and is the foundation for systems like BERT, RoBERTa, and GPT.

3.7.1. Transformers vs. Sentence Transformers: A Critical Distinction

Understanding the two primary ways to use transformers is essential to understanding our methodology.

- **Standard Transformers (e.g., BERT):** A standard transformer model (like `bert-base-uncased`) is trained on tasks like "Masked Language Modeling" (guessing a hidden word) and "Next Sentence Prediction." When it processes a review, its primary output is a set of **contextual word embeddings**—one 768-dimension vector for *each word* in the review. It does *not* natively output one single, meaningful vector for the *entire review*.
- **Sentence Transformers (e.g., SBERT):** A Sentence Transformer is a standard transformer model that has been **specifically fine-tuned** on a secondary task. Using a Siamese network structure, it is trained on massive datasets of sentence pairs to understand semantic similarity. The goal of this training is to produce a *single, fixed-size vector* for the entire input sentence. In this new vector space, semantically similar sentences are mathematically close to each other, and dissimilar sentences are far apart.

This distinction is the most important factor in our modeling choice.

3.7.2. Overview of Common Foundational Transformer Models

To provide context, the following table summarizes the most common "foundational" models in the transformer ecosystem. These are typically used as a *base* for fine-tuning.

Model	Description	Primary Use Case	Caveat for Our Project (Feature Extraction)
BERT	Bidirectional Encoder Representations from Transformers. The original "game-changer," it reads the entire text at once (bidirectionally).	**Fine-tuning** for classification, question answering, and named-entity recognition.	Not trained to produce meaningful *sent ence-level* embeddings "out-of-the-box."
RoBERTa	A "Robustly Optimized BERT Pretraining Approach." It's a BERT model trained on more data, for longer, with a smarter training process.	**Fine-tuning.** It generally outperforms BERT on the same tasks.	Same as BERT. Its output vectors are for tokens, not for the whole sentence.
GPT (Series)	Generative Pre-trained Transformer. A "decoder-only" model that reads text from left to right (autoregressive) .	**Text Generation.** Famously used for chatbots, summarization, and creative writing.	Not designed for classification. Its embeddings are for predicting the *next word*, not for summarizing the *full text*.
T5	Text-to-Text Transfer Transformer. An "encoder-	**Fine-tuning** for translation, summarization,	Overkill for feature extraction. It's designed to be

Model	Description	Primary Use Case	Caveat for Our Project (Feature Extraction)
	decoder" model that frames every NLP task as a text-to-text problem (e.g., "classify: [review text]").	and question answering.	fine-tuned to *generate* a class label as text (e.g., "5 stars").

3.7.3. Transformer-Based Strategies for Rating Prediction

There are two main strategies to use these models for our task. Our choice was driven by project goals and computational resources.

Approach 1: Fine-Tuning (Not Used in This Project)

This is the most common "heavy-duty" approach.

- **How it works:** One would load a standard transformer (like `distilbert-base-uncased` or `roberta-base`), add a new 5-neuron classification layer on top, and then re-train the entire model on our review texts. The model's internal weights would be "fine-tuned" to become an expert at our specific 1-5 star rating task.
- **Pros:** This method typically yields the highest possible predictive accuracy (state-of-the-art).
- **Cons:** It is computationally **very expensive** and time-consuming, requiring a powerful GPU and a complex training pipeline. It is a full deep-learning task, not a classical ML task.

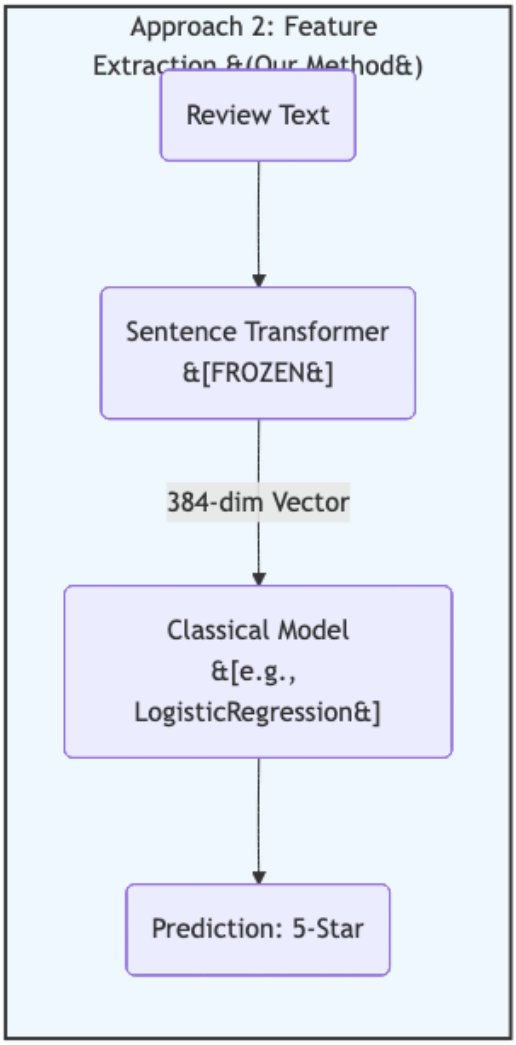
Approach 2: Feature Extraction (Our Chosen Method)

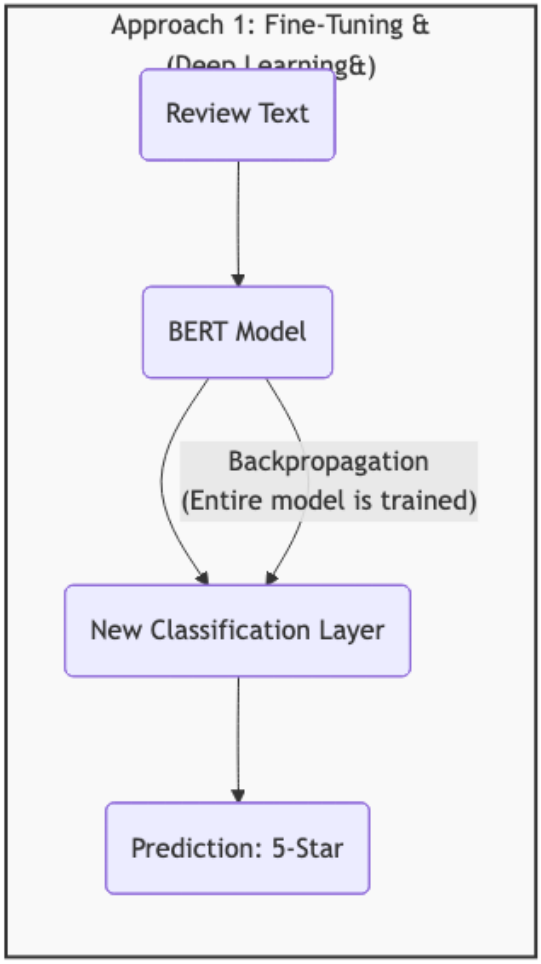
This is a much faster and more flexible "hybrid" approach.

- **How it works:** We use a **Sentence Transformer** (not a standard one) as a simple conversion tool. We do not train it. It is used "as-is" to convert each review in our dataset into a single, high-quality vector.
- **Our Pipeline:**
    1. **Review Text:** "The product was great, but the shipping was slow."

2. **Sentence Transformer:** `all-MiniLM-L6-v2` processes the text.
3. **Feature Vector:** A single 384-dimension vector `[0.12, -0.45, ..., 0.88]` is produced.
4. **Classical Model:** This vector (along with our other numerical features) is fed into a fast, robust classifier like `LGBMClassifier` or `LogisticRegression`.

- **Pros:**
  - **Fast:** We only need to compute the embeddings once.
  - **Flexible:** We can test multiple classical classifiers (LGBM, SVM, etc.) on these rich features.
  - **Combines Features:** This pipeline easily allows us to combine the 384-dim text embedding with our other hand-crafted numerical features (word count, sentence length, etc.).
  - **Computationally Cheap:** Training the `LGBMClassifier` on, e.g., 50,000 rows of 384 features is fast and can be done on a CPU.





3.7.4. Why `bert-base-uncased` is Unsuitable for Our Approach

A common first thought is to use the `bert-base-uncased` model for feature extraction. This is a trap that leads to poor performance.

1. **The `[CLS]` Token is Not a Good Embedding:** `bert-base-uncased` outputs a special `[CLS]` (classification) token at the beginning of each sentence. It is tempting to use this token's 768-dim vector as the "sentence embedding." However, this vector was *only* pre-trained on the very simple "Next Sentence Prediction" task. It **does not contain rich semantic meaning** for the whole sentence and performs very poorly as a general-purpose embedding.

2. **Averaging Tokens "Muddies" the Signal:** The second idea is to take all the word embeddings from BERT and average them. This suffers from the **exact same problem as our Word2Vec averaging**: important keywords like "terrible" and "amazing" are averaged together, canceling each other out and resulting in a "muddy," neutral vector.

We require a model that was *explicitly trained* to solve this problem, which is what Sentence Transformers do.

3.7.5. Recommended Models and Final Choice

For our feature extraction pipeline, we selected a model from the `sentence-transformers` library. These models are hosted on Hugging Face and are designed for this exact purpose.

Recommended Models for Feature Extraction:

Model Name (Hugging Face)	Output Dimensions	Key Benefit
`all-MiniLM-L6-v2`	384	**Excellent Balance.** Very fast, small vector size, and top-tier performance.
`all-mpnet-base-v2`	768	**Highest Accuracy.** Often considered the best all-around model, but is significantly

Model Name (Hugging Face)	Output Dimensions	Key Benefit
		slower and produces larger vectors.

Our Choice: `all-MiniLM-L6-v2`

For this project, we selected **`all-MiniLM-L6-v2`** as our embedding model. It represents the optimal balance of performance and efficiency for our pipeline:

1. **Speed:** It is a "Mini" model, making it very fast at converting our review texts into embeddings.
2. **Size:** It outputs a 384-dimension vector. This is a "sweet spot" for classical classifiers like LGBM, as it's information-dense but not so large that it causes the "curse of dimensionality" or slow training.
3. **Performance:** Despite its small size, it was trained on over 1 billion sentence pairs and achieves performance very close to the larger `mpnet` model.

This allows us to leverage the power of transformers in a fast, efficient, and highly effective hybrid pipeline.

3.8. Overview of Classifier Models

To ensure a comprehensive and fair comparison, we selected four distinct classifiers for our experiments. These models represent two different "families" of algorithms: **Linear Models** (which are traditionally strong on sparse, high-dimensional TF-IDF data) and **Tree-Based Ensembles** (which are state-of-the-art for dense, tabular-style data like embeddings).

This variety allows us to test our hypotheses (Section 4.2) about which classifier type is best suited for each of our feature engineering pipelines.

3.8.1. Classifier Summary Table

Model	Model Family	How it Works	Best Suited For…	Key Hyperparameters
		Predicts class	**Sparse, High-**	`C` (regularizati

Model	Model Family	How it Works	Best Suited For…	Key Hyperparameters
`LogisticRegression`	Linear Model	probability by fitting a linear equation to the features via a sigmoid function.	**Dimensional Data (TF-IDF)**. Fast, highly interpretable, and robust to overfitting due to built-in regularization.	on strength), `solver`
`LinearSVC`	Linear Model (SVM)	Finds the optimal "hyperplane" that creates the widest possible margin between classes.	**Sparse, High-Dimensional Data (TF-IDF)**. Very effective and memory-efficient for text classification.	`C` (regularization strength)
`RandomForestClassifier`	Ensemble (Bagging)	Builds a "forest" of many independent decision trees and uses a majority vote to predict.	**Dense Data (Embeddings)**. Highly robust to overfitting, easy to tune, and good at finding non-linear patterns.	`n_estimators` (number of trees), `max_depth` (tree complexity)

Model	Model Family	How it Works	Best Suited For…	Key Hyperparameters
`LGBMClassifier`	Ensemble (Boosting)	Builds trees *sequentially*, where each new tree is trained to correct the errors of the previous ones.	**Dense Data (Embeddings)**. State-of-the-art performance and speed. Excellent at finding complex, non-linear patterns.	`learning_rate`, `num_leaves` (tree complexity, `subsample`

3.8.2. Classifier Descriptions

LogisticRegression

This is a fundamental linear classifier that remains one of the most effective baselines for text classification. It works by finding a linear combination of the features (e.g., TF-IDF scores) that best separates the classes. Its built-in **L2 regularization** (controlled by the `C` hyperparameter) makes it highly effective at handling very wide, sparse feature sets without overfitting, which is why it's a top candidate for our `Classical (TF-IDF)` pipeline.

LinearSVC (Support Vector Classifier)

This is another high-performance linear model based on the Support Vector Machine (SVM) algorithm. While `LogisticRegression` focuses on probability, `LinearSVC` focuses on geometry. It attempts to find the single best "hyperplane" (a line, in high dimensions) that creates the **maximum possible margin** between the different classes. Like Logistic Regression, it is extremely efficient and effective on sparse `TF-IDF` data and is a standard for text classification tasks.

RandomForestClassifier

This is an **ensemble** model that operates on the principle of "wisdom of the crowd." It builds a large number of individual decision trees (a "forest") using a technique called *bagging*. Each tree is trained on a slightly different, random

subset of the training data and features. The final prediction is determined by a majority vote from all the trees. This averaging process makes `RandomForest` highly robust and resistant to overfitting. It is particularly good at finding complex, non-linear patterns in **dense, lower-dimensional data** like our `Embedding` vectors.

LGBMClassifier (LightGBM)

This is a state-of-the-art **gradient boosting** model. Unlike `RandomForest`, which builds trees in parallel, a boosting model builds them *sequentially*. Each new tree is explicitly trained to correct the mistakes made by the collection of trees that came before it. `LightGBM` is a specific implementation known for its **high speed and top-tier accuracy**. It is often the winning model in machine learning competitions involving tabular or dense vector data. We expect this model to be our strongest contender for the `Embedding` and `Transformer` feature sets.

4. Expected Results & Hypotheses

Before presenting the results, it is useful to establish a hypothesis about the expected performance of our different approaches. This allows us to assess whether the outcomes are in line with theory or if there are surprising findings.

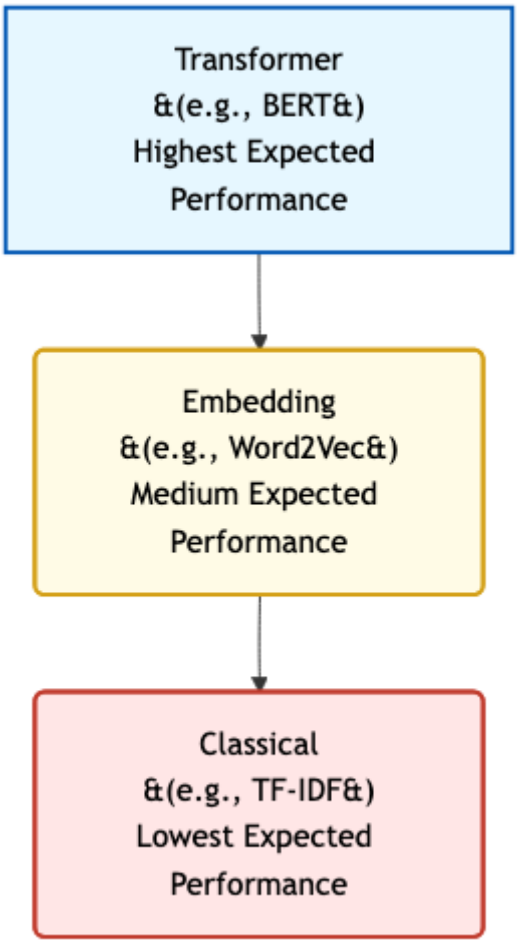
4.1. Expected Performance Hierarchy

We hypothesize the following performance ranking, from best to worst:

Transformer > Embedding > Classical (TF-IDF)

- **Transformer (Highest Expectation):** Transformer models are context-aware. They can understand the difference between “The shipping was fast, but the product is *bad*” and “The product is good, but the shipping is *bad*.” This ability to grasp context and long-range dependencies is expected to give it a significant edge over other methods.
- **Embedding (Middle Expectation):** Embedding-based approaches capture semantic meaning, understanding that “great” and “excellent” are similar. Our use of **TF-IDF weighted averaging** and **FastText’s** `get_sentence_vector` should make this approach significantly stronger than a naive implementation. However, since it still aggregates word vectors, it loses some of the word order and contextual information that transformers retain.
- **Classical (Lowest Expectation):** While a strong baseline, TF-IDF is the most simplistic model. It has no semantic understanding and is purely

based on keyword identification. We expect it to perform well but to be surpassed by the more advanced methods that understand word meaning and context.



4.2. Expected Classifier Performance

- **For TF-IDF (Sparse Data):** We expect **linear models** like `LogisticRegression` and `LinearSVC` to perform very well. They are highly effective and computationally efficient on high-dimensional, sparse data.
- **For Embedding & Transformer (Dense Data):** We expect **tree-based models** like `GradientBoostingClassifier` and `RandomForestClassifier` to excel. These models are adept at finding complex, non-linear patterns in dense, meaningful feature spaces.

5. Understanding Model Explanations with SHAP

To understand *why* our models make certain predictions, we use SHAP (SHapley Additive exPlanations). We generate two types of explanations, depending on the model.

5.1. For Classical (TF-IDF) Models: Standard SHAP

The SHAP plots for TF-IDF models show the importance of each word (or n-gram) as a feature.

- **Bar Plot:** Shows the average impact of each feature on the model's output magnitude.
- **Beeswarm Plot:** More informative. Each dot is a single prediction. A positive SHAP value pushes the prediction towards a higher rating, while a negative value pushes it towards a lower rating. The color indicates the feature's value (red for high presence/score).

5.2. For Embedding and Transformer Models: Word-Level SHAP

SHAP values on dense embedding dimensions (e.g., `dim_57`) are not interpretable. To solve this, we implemented **word-level SHAP** using a perturbation-based approach.

- **How it Works:** The explainer creates many versions of a sentence, each with some words masked (removed). It observes how the model's prediction changes as words are removed to determine each word's contribution.
- **Interpretation:** The plots directly show which **words** in the original text pushed the prediction higher or lower, allowing us to peek inside the complex models.

6. Experiment Results and Detailed Analysis

This section presents the comprehensive results from our two primary experimental setups: the main **multi-class classification** task and a simplified **binary classification** task.

Our central objective, as defined in the executive summary, is the 1-5 star multi-class problem. We will first detail the performance of the `Classical (TF-IDF)` and `Embedding (Word2Vec, FastText)` approaches for this complex task.

Following the multi-class analysis, we will present the results from a secondary set of experiments. For this, we reframed the problem as a high-value binary classification (Negative vs. Positive) by mapping 1/2-star and 4/5-star reviews to a binary target. This simplified task allows us to evaluate the models on pure sentiment polarity and establish a strong performance benchmark for sentiment flagging.

6.1. General Experiment Setup

All experiments were conducted using a consistent stratified 80% train / 20% test split to maintain the original class distributions in both sets. The primary optimization metric for all studies was the **weighted F1-score**, as it provides the best balance for this imbalanced, multi-class problem.

Hyperparameter optimization for all models was performed using **Optuna** with the following configuration to balance thoroughness and computational efficiency: * **Number of Max Trials:** 30 trials per model. * **Early Stopping Trials:** 10 trials without improvement would trigger early stopping. * **Timeout:** Each Optuna study had a maximum runtime (typically 20-60 minutes).

A key difference between the two primary experiments was the dataset size, which was chosen based on the task's complexity and computational requirements:

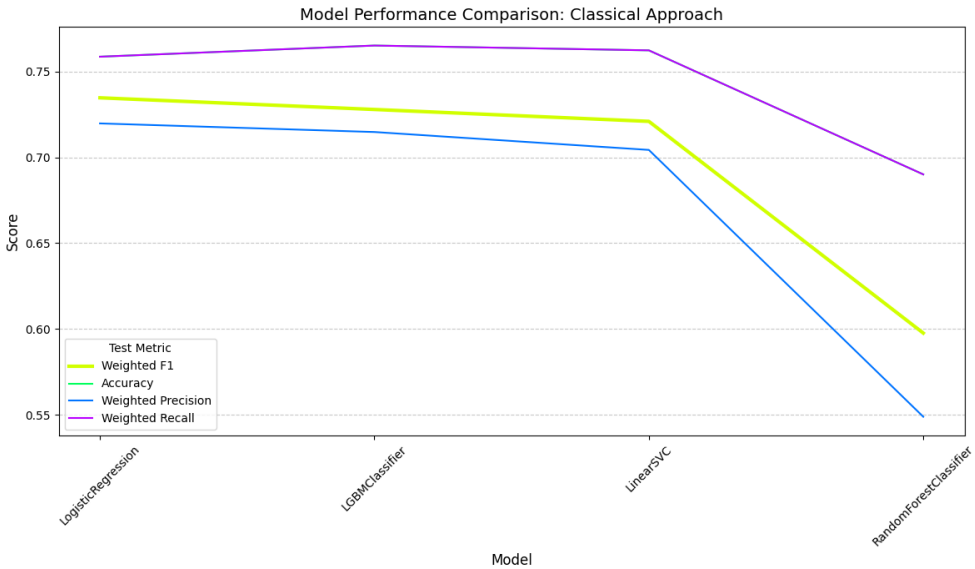
- **Multi-Class Experiments (Sections 6.2 - 6.4):** These experiments involved our primary, most complex objective (1-5 stars) and included the computationally intensive `Embedding` and `Transformer` approaches. To ensure it was feasible to run comprehensive Optuna tuning across all three paradigms, we used a representative subset of **50,000 training samples and 10,000 test samples**.
- **Binary Classification Experiments (Section 6.6):** This task was simpler (Positive/Negative) and, as the results show, the highly efficient `Classical (TF-IDF)` models were dominant. Given the lower computational cost, we were able to leverage a larger dataset to build a more robust final model. These experiments were conducted on **100,000 training samples and 20,000 test samples**.

6.2. Classical Approach Results (TF-IDF)

6.2.1. Comparison of All Classical Models

The classical approach, utilizing TF-IDF for text vectorization, served as our robust baseline. The following table and plot summarize the performance of the best run for each classifier tested.

Model	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
Logistic Regression	0.7347	0.7587	0.7198	0.7587	0.842	58.94
LGBMClassifier	0.7279	0.7652	0.7148	0.7652	0.8602	29.64
LinearSVC	0.721	0.7624	0.7044	0.7624	0.9464	40.25
RandomForestClassifier	0.5976	0.6901	0.5489	0.6901	0.6108	59.65



classical Comparison Plot

Model Performance Summary

This table summarizes the performance of the four classifiers on the multi-class TF-IDF task.

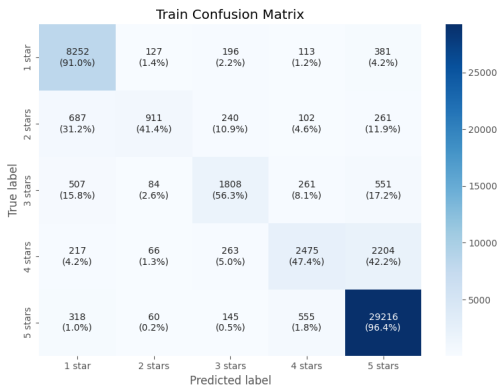
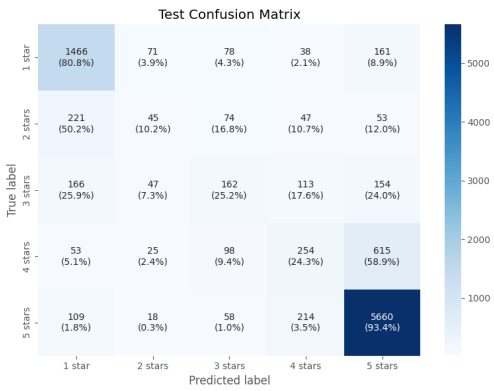
- **Top Performers:** `LogisticRegression` achieves the highest weighted F1-score (0.7347). `LGBMClassifier` is a very close second in performance (0.7279 F1-score) but was significantly **twice as fast to train** (29.64s) than `LogisticRegression` (58.94s). `LinearSVC` also performs well (0.721 F1) but shows the **highest signs of overfitting**, with a training F1-score of 0.9464.
- **Lower Performer:** `RandomForestClassifier` performs considerably worse than the other models across all test metrics (0.5976 F1) and was the slowest to train (59.65s).

Conclusion: While `LGBMClassifier` was the most efficient, **`LogisticRegression` is the recommended winner** for this category. It offers the **best predictive performance (highest weighted F1)** and a more balanced train/test F1-score, indicating better generalization than the highly overfit `LinearSVC`.

6.2.2. Analysis of the Best Classical Model: `LogisticRegression`

The `LogisticRegression` emerged as the top performer, achieving a weighted F1-score of **0.7347**.

Evaluation Metrics

Confusion Matrix (Train Set)	Confusion Matrix (Test Set)
	

Classification Report (Train Set)

	precision	recall	f1-score	support
1 star	82.7%	91.0%	86.6%	9,069
2 stars	73.0%	41.4%	52.8%	2,201
3 stars	68.2%	56.3%	61.7%	3,211
4 stars	70.6%	47.4%	56.7%	5,225
5 stars	89.6%	96.4%	92.9%	30,294
accuracy			85.3%	50,000
macro avg	76.8%	66.5%	70.1%	50,000
weighted avg	84.2%	85.3%	84.2%	50,000

Classification Report (Test Set)

	precision	recall	f1-score	support
1 star	72.8%	80.8%	76.6%	1,814
2 stars	21.8%	10.2%	13.9%	440
3 stars	34.5%	25.2%	29.1%	642
4 stars	38.1%	24.3%	29.7%	1,045
5 stars	85.2%	93.4%	89.1%	6,059
accuracy			75.9%	10,000
macro avg	50.5%	46.8%	47.7%	10,000
weighted avg	72.0%	75.9%	73.5%	10,000

Confusion Matrix and Classification Report Analysis

The confusion matrices and classification reports provide a detailed view of the model's performance on both training and test data, revealing its strengths and weaknesses in predicting star ratings.

Training Performance

- **Overall High Accuracy:** The model achieves a high overall accuracy (85.3%) and weighted F1-score (84.2%) on the training set.
- **Strong on Extremes:** It excels at identifying **1-star** (F1: 86.6%, Recall: 91.0%) and particularly **5-star** reviews (F1: 92.9%, Recall: 96.4%). This aligns with the confusion matrix showing high true positive rates for these classes.
- **Weaker on Mid-Ratings:** Performance significantly drops for the intermediate ratings: **2-star** (F1: 52.8%, Recall: 41.4%), **3-star** (F1: 61.7%, Recall: 56.3%), and **4-star** (F1: 56.7%, Recall: 47.4%). The confusion matrix confirms this, showing substantial misclassifications into adjacent or extreme categories (e.g., 4-stars often predicted as 5-stars).
- **Imbalance Impact:** The high weighted average F1 is heavily influenced by the model's strong performance on the dominant 5-star class (support: 30,294). The macro average F1 (70.1%) gives a better sense of the lower average performance across all classes treated equally.

Test Performance & Generalization

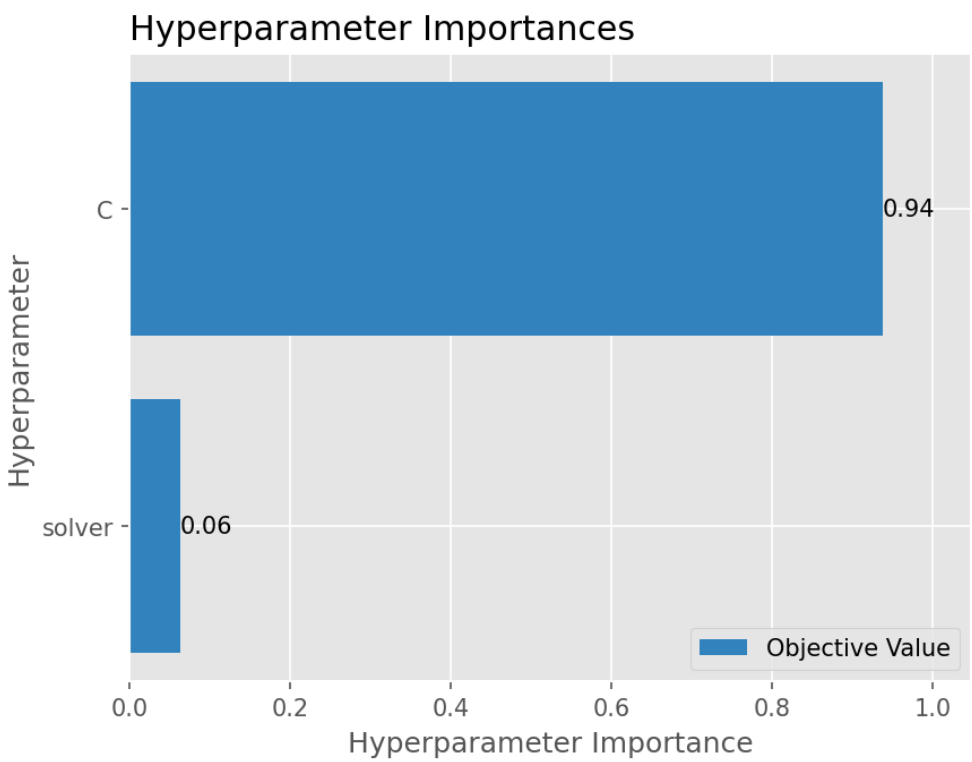
- **Accuracy Drop:** Overall accuracy decreases to 75.9% and the weighted F1-score falls to 73.5% on the test set, indicating overfitting.
- **Good Generalization for 5-Stars:** The model maintains good performance for **5-star** reviews (F1: 89.1%, Recall: 93.4%), showing it generalizes well for strongly positive sentiment.
- **Performance Collapse for Mid/Low Ratings:** Generalization is very poor for other classes:
  - **1-star:** F1 drops to 76.6% (Recall: 80.8%).
  - **2-star:** Performance collapses (F1: 13.9%, Recall: **10.2%**). The confusion matrix shows most 2-star reviews are misclassified as 1-star.
  - **3-star:** Performance is very weak (F1: 29.1%, Recall: **25.2%**). The confusion matrix shows widespread confusion, often misclassified as 1-star or 5-star.

    - **4-star:** Performance is also very weak (F1: 29.7%, Recall: **24.3%**). The confusion matrix shows a strong tendency to misclassify these as 5-star.
- **Imbalance Impact Magnified:** The significant gap between the weighted F1 (73.5%) and the extremely low macro F1 (47.7%) on the test set highlights the model's failure. It performs passably on the most common class (5-stars) but poorly on almost all others when generalizing.

Summary

The model demonstrates **significant overfitting**. While it learned to classify the training data reasonably well (especially the dominant 5-star and distinct 1-star classes), it fails to generalize effectively to unseen data for the **2, 3, and 4-star categories**. The extremely low recall and F1-scores for these classes on the test set indicate the model struggles to identify the nuanced features distinguishing these intermediate ratings, often defaulting to classifying them as 1-star or 5-star. Addressing this overfitting and improving the discrimination between adjacent and mid-range classes is crucial for better real-world performance.

Optuna Hyperparameter Importance (Logistic Regression)



Optuna Importance Plot

This plot illustrates the relative importance of the hyperparameters tuned for the Logistic Regression model, based on how much changes in each hyperparameter

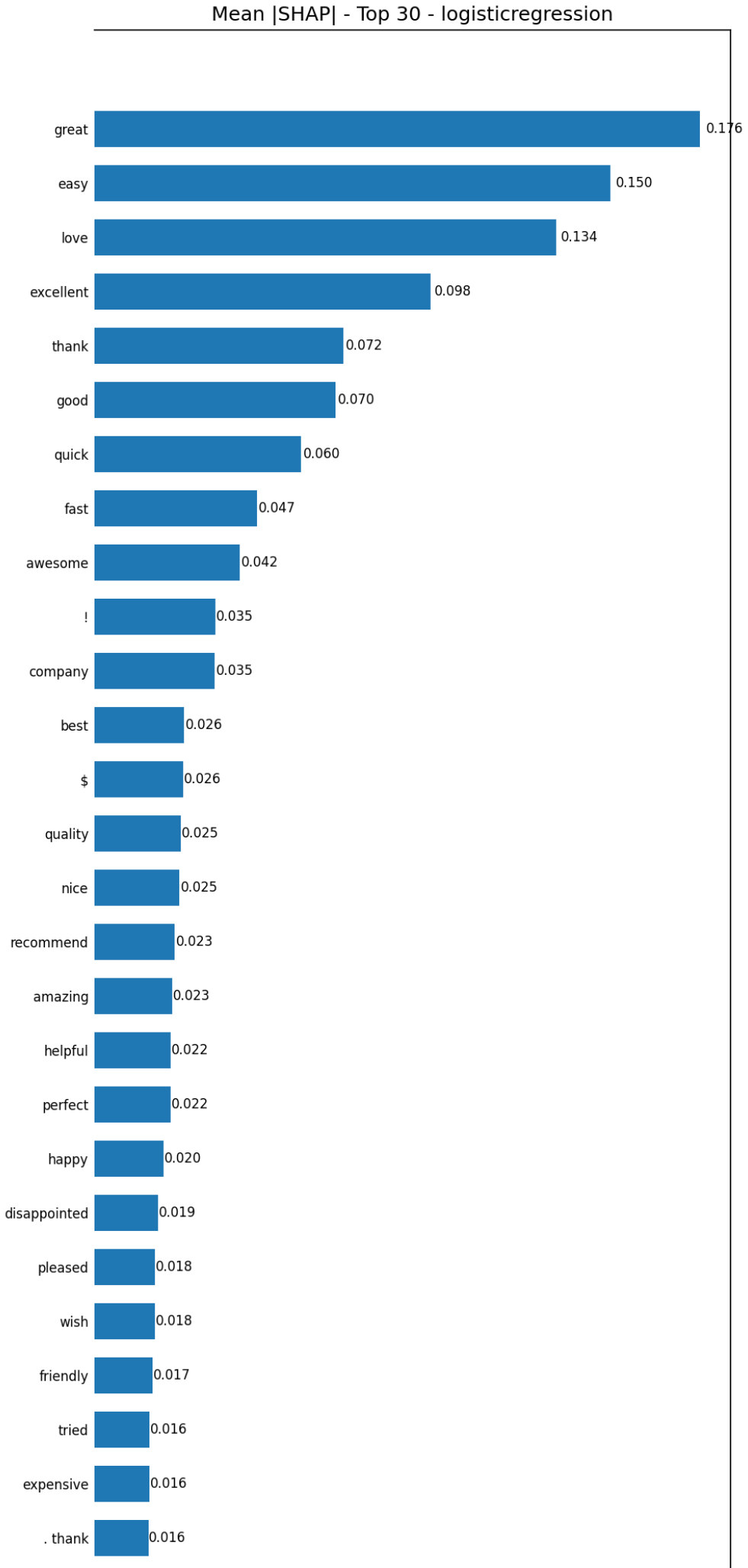
affected the final objective value (e.g., weighted F1-score) during the Optuna optimization process.

- **`C` (Inverse Regularization Strength):** This hyperparameter shows **overwhelming importance (0.94)**. This indicates that the performance of the Logistic Regression model was **highly sensitive** to the level of regularization applied. Finding the optimal value for `C` was the most critical factor in achieving the best results during the tuning study. Variations in `C` caused significant changes in the model's F1-score.
- **`solver`:** The choice of solver (`saga` vs. `lbfgs`) demonstrates **very low importance (0.06)**. This suggests that, while different solvers might have affected training time or allowed different penalties, the actual *peak performance* achieved was **not strongly dependent** on which solver was used, provided the `C` parameter was appropriately tuned for that solver.

In essence, for this specific tuning run, correctly setting the regularization strength (`C`) was far more crucial for maximizing the model's predictive performance than the choice between the `saga` and `lbfgs` optimization algorithms.

In conclusion, the bar plot quantifies the overall importance hierarchy observed qualitatively in the beeswarm plots. It confirms that the Logistic Regression model relies heavily on a vocabulary of strong positive sentiment words to make its predictions, with negative and neutral/process-related terms having a secondary, though still measurable, overall impact.

Model Interpretability (SHAP)


Mean |SHAP| - Top 30 - logisticregression
great 0.176
easy 0.150
love 0.134
excellent 0.098
thank 0.072
good 0.070
quick 0.060
fast 0.047
awesome 0.042
! 0.035
company 0.035
best 0.026
$ 0.026
quality 0.025
nice 0.025
recommend 0.023
amazing 0.023
helpful 0.022
perfect 0.022
happy 0.020
disappointed 0.019
pleased 0.018
wish 0.018
friendly 0.017
tried 0.016
expensive 0.016
. thank 0.016

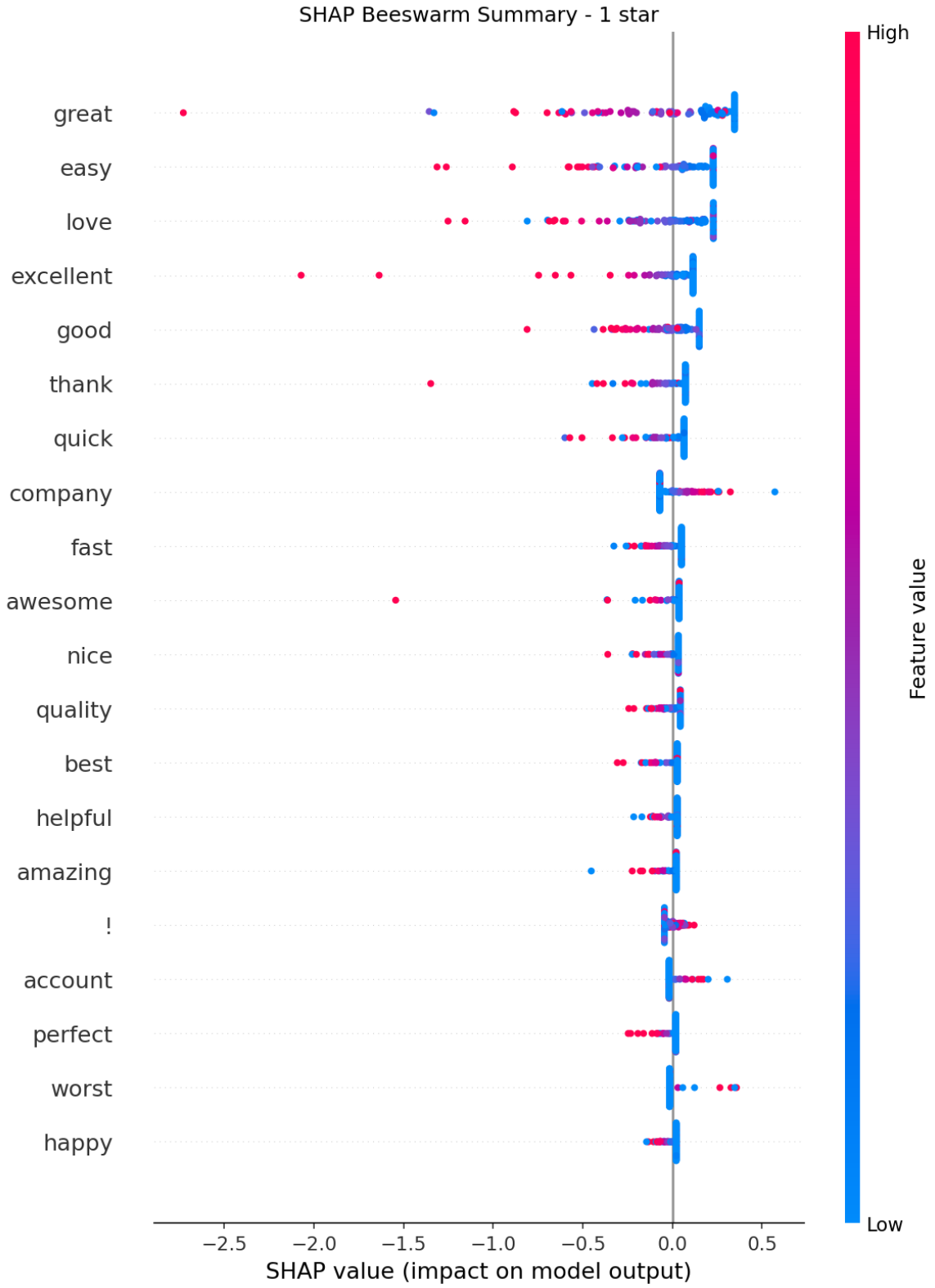
SHAP Bar Plot

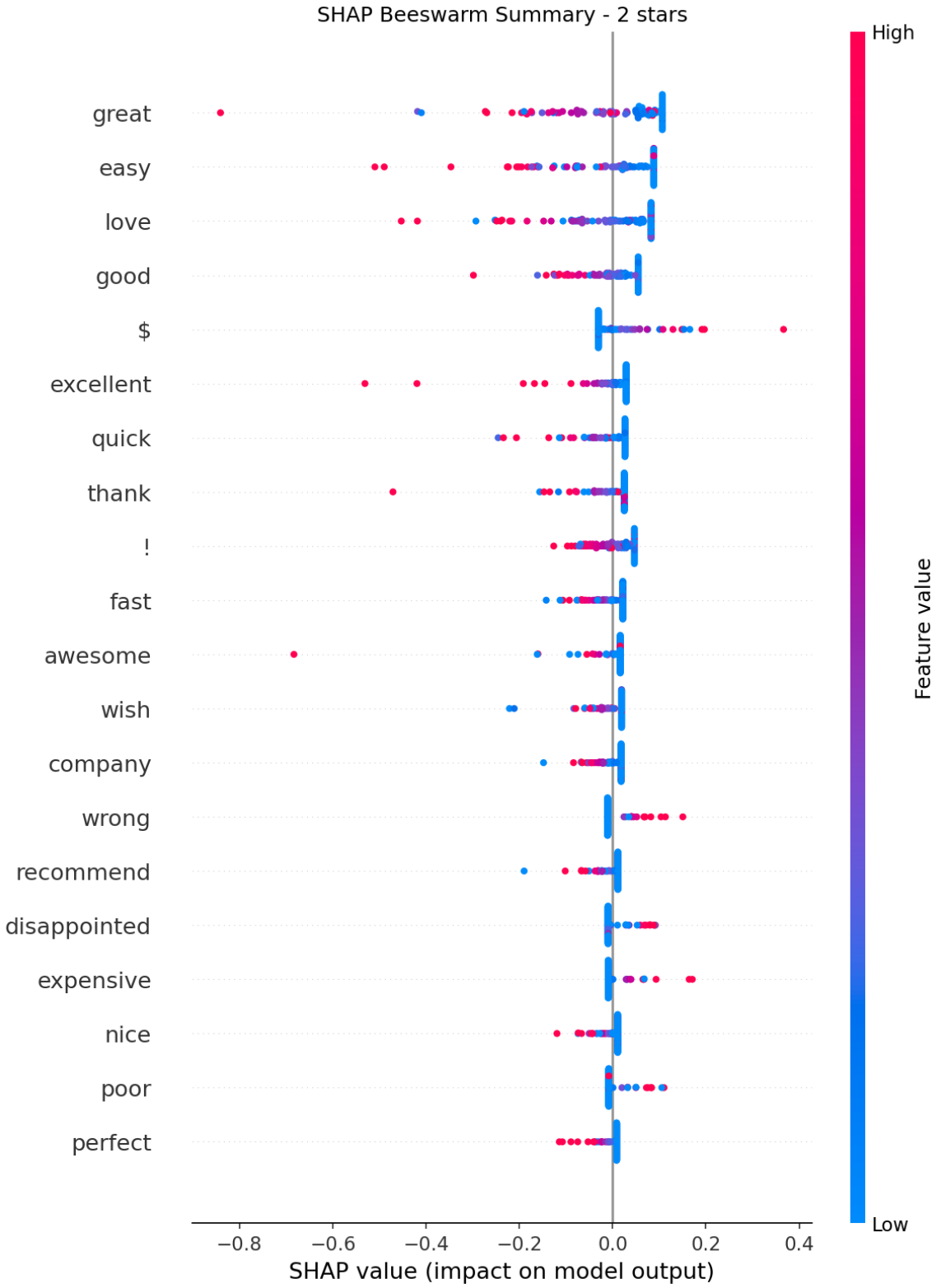
SHAP Analysis Summary (Logistic Regression - Bar Plot)

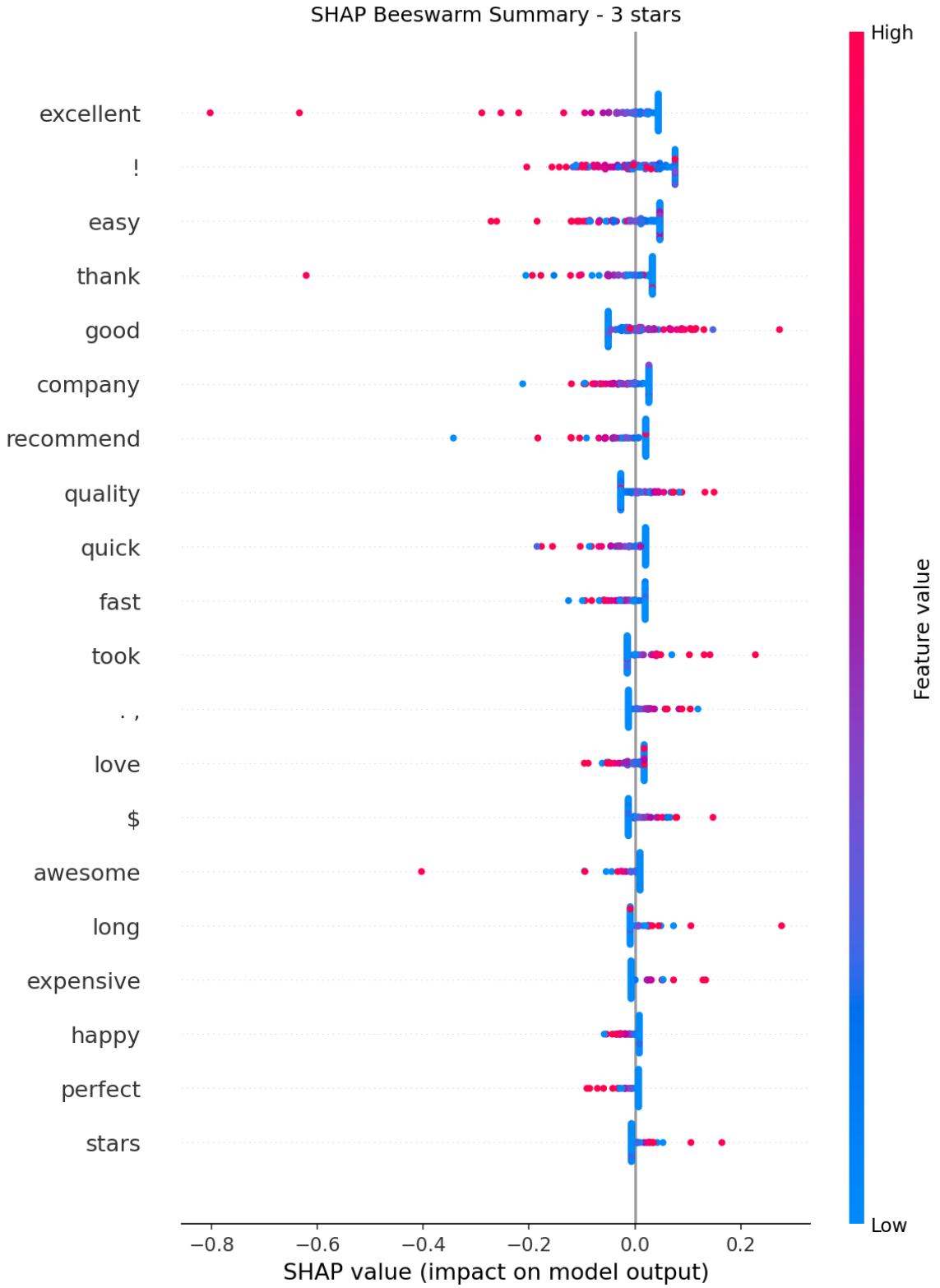
The SHAP bar plot displays the **top 30 features** ranked by their **mean absolute SHAP value** across all predictions. This metric represents the average magnitude of impact each feature (word) has on the model's output, regardless of whether it pushes the prediction higher or lower. It essentially ranks features by their overall predictive importance.

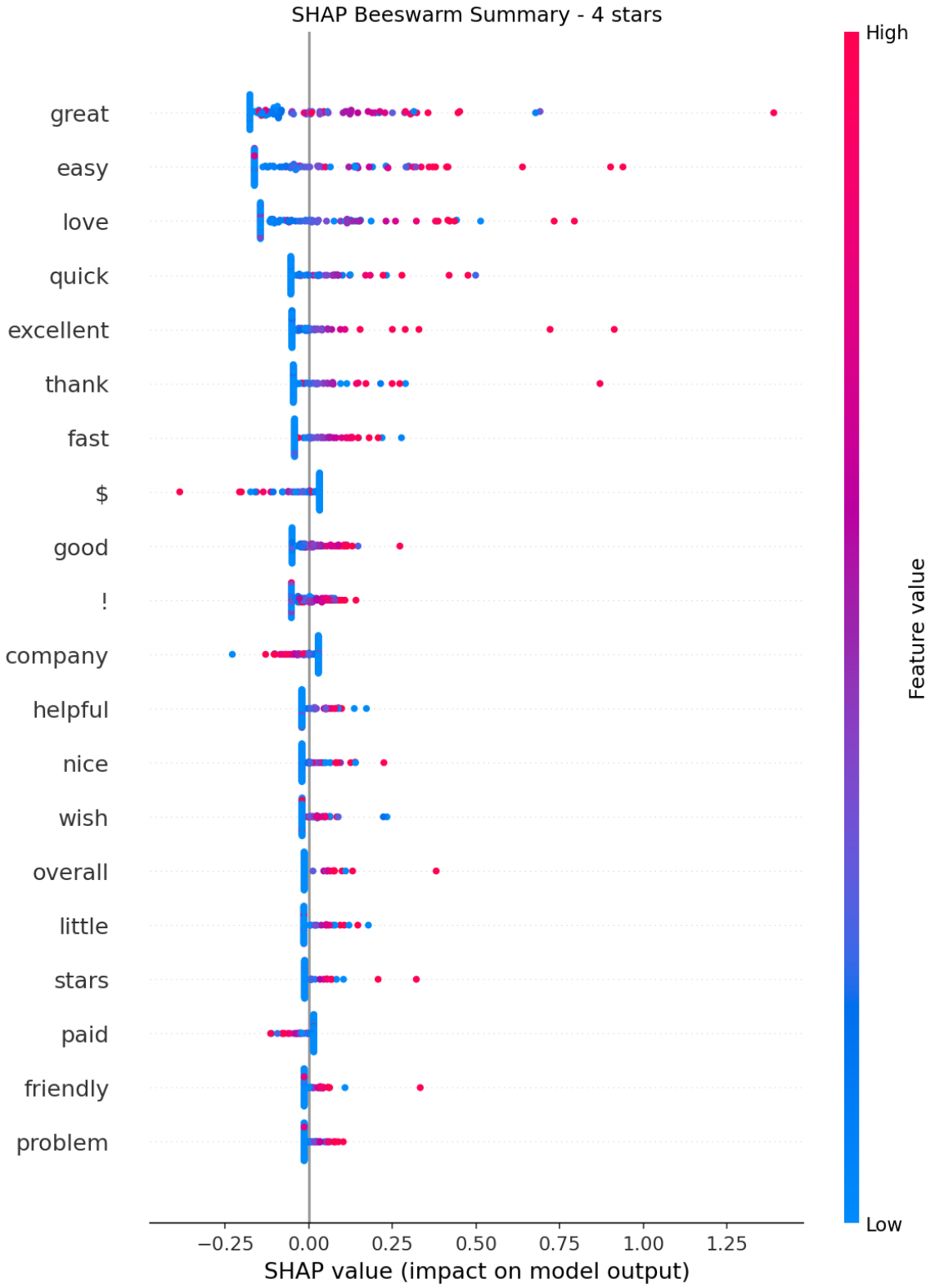
- **Top Influencers:** Confirming the observations from the beeswarm plots, strongly positive words like `great`, `easy`, `love`, and `excellent` are clearly the most impactful features overall. Words like `thank`, `good`, `quick`, `fast`, and `awesome` follow, indicating their consistent relevance in determining the predicted rating.
- **Consistency with Beeswarm Plots:** The features dominating this bar plot are the same ones that showed the largest positive SHAP values for 5-star predictions and the largest negative SHAP values for 1-star predictions in the beeswarm plots. This reinforces that these strong sentiment words are the primary drivers of the model's predictions, particularly for the extreme ratings.
- **Mid-Impact Features:** Features like `company`, `$`, `quality`, `nice`, `recommend`, `amazing`, `helpful`, `perfect`, and `happy` show moderate mean absolute SHAP values. This aligns with the beeswarm plots where these words often had noticeable, though less extreme, impacts across various rating classes, sometimes pushing towards positive ratings and sometimes away from negative ones.
- **Lower-Impact Negatives:** Some negative words like `disappointed` and `expensive` appear in the top 30 but have lower mean absolute SHAP values compared to the top positive words. This might suggest that while important for identifying lower ratings (as seen in the 1-star and 2-star beeswarm plots), their overall average impact across *all* predictions is slightly less than that of the strongest positive terms in this specific model's view.

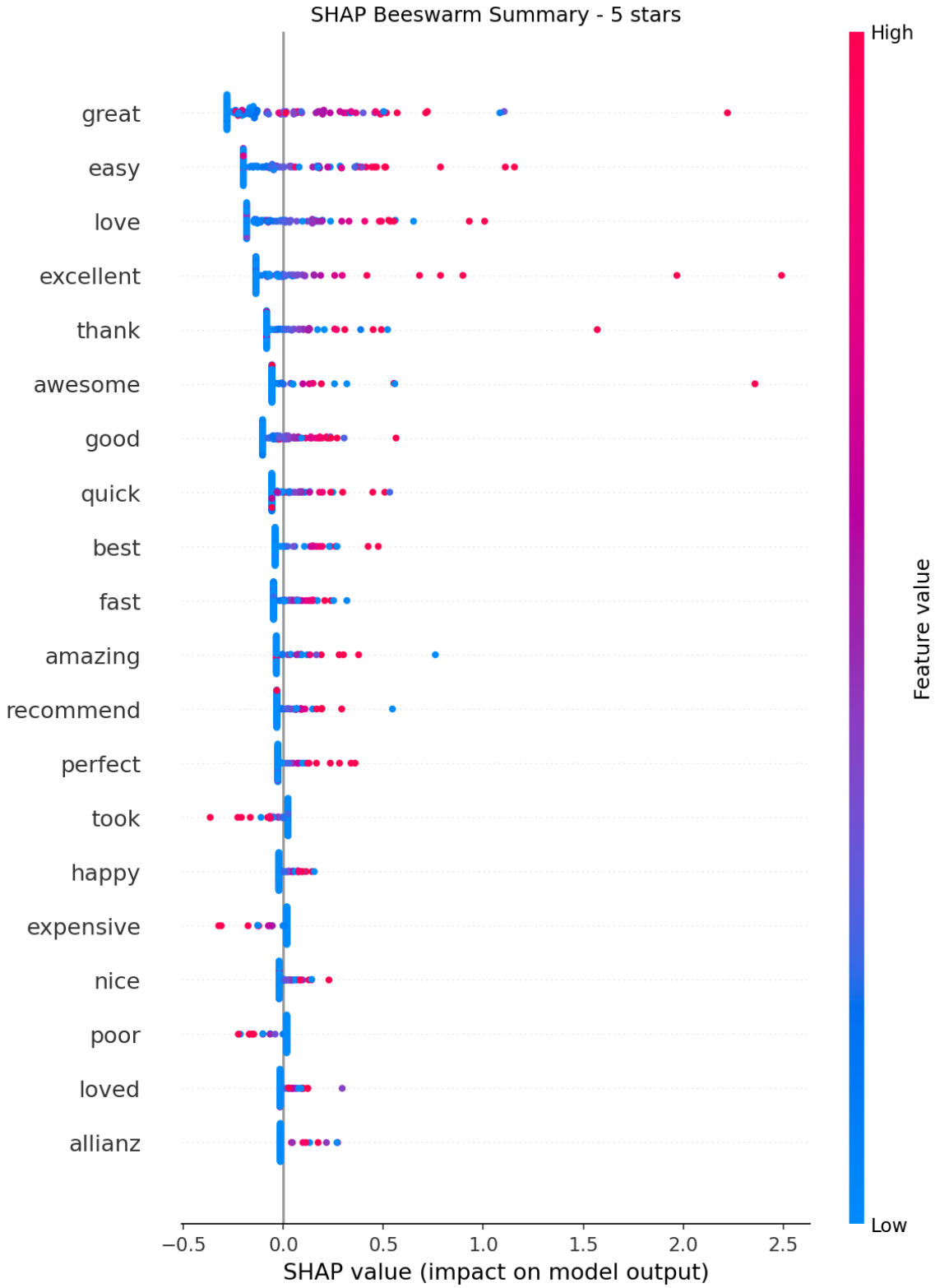
SHAP Bar Plot and Beeswarm Plots (Per Class)


SHAP Beeswarm Summary - 1 star
great
easy
love
excellent
good
thank
quick
company
fast
awesome
nice
quality
best
helpful
amazing
!
account
perfect
worst
happy
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
SHAP value (impact on model output)
High
Feature value
Low


SHAP Beeswarm Summary - 2 stars
great
easy
love
good
$
excellent
quick
thank
!
fast
awesome
wish
company
wrong
recommend
disappointed
expensive
nice
poor
perfect
−0.8
−0.6
−0.4
−0.2
0.0
0.2
0.4
SHAP value (impact on model output)
High
Feature value
Low


SHAP Beeswarm Summary - 3 stars
excellent
!
easy
thank
good
company
recommend
quality
quick
fast
took
. ,
love
$
awesome
long
expensive
happy
perfect
stars
−0.8
−0.6
−0.4
−0.2
0.0
0.2
SHAP value (impact on model output)
High
Feature value
Low


SHAP Beeswarm Summary - 4 stars
great
easy
love
quick
excellent
thank
fast
$
good
!
company
helpful
nice
wish
overall
little
stars
paid
friendly
problem
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
SHAP value (impact on model output)
High
Feature value
Low



The SHAP beeswarm plots reveal that the Logistic Regression model, trained on TF-IDF features, largely relies on intuitive word associations to predict star ratings.

- **Strong Sentiment Dominance:** Words with strong positive (`great` , `easy` , `love` , `excellent` , `perfect`) or negative (`worst` , `poor` , `disappointed` ,

`problem`) connotations consistently emerge as the most impactful features across all rating classes.

- **Predicting Extremes (1 & 5 Stars):** The model strongly associates the *presence* (high TF-IDF value) of highly positive words with 5-star predictions and pushes predictions *away* from 1-star when they are present. Conversely, the presence of strongly negative words like `worst` drives predictions towards 1-star and strongly away from 5-stars. The absence of strong sentiment in either direction is also a key factor for these extreme ratings.
- **Nuance in Mid-Ratings (2-4 Stars):** Predictions for 2, 3, and 4 stars show more subtle influences.
  - **2-Star:** Driven by moderately negative terms (`wrong`, `disappointed`, `expensive`, `$`) and the absence of strong positives.
  - **3-Star:** Characterized by the *lack* of strong sentiment words or a mix of mild positive/negative signals. Fewer words strongly push *towards* this neutral rating; its prediction often results from the absence of decisive language. Words related to process or neutral aspects (`company`, `quality`, `took`) show varied impacts.
  - **4-Star:** Clearly driven by positive language (`great`, `easy`, `good`, `helpful`, `friendly`), but potentially moderated by the presence of minor negative points (`problem`, `$`) compared to the overwhelming positivity required for 5 stars.

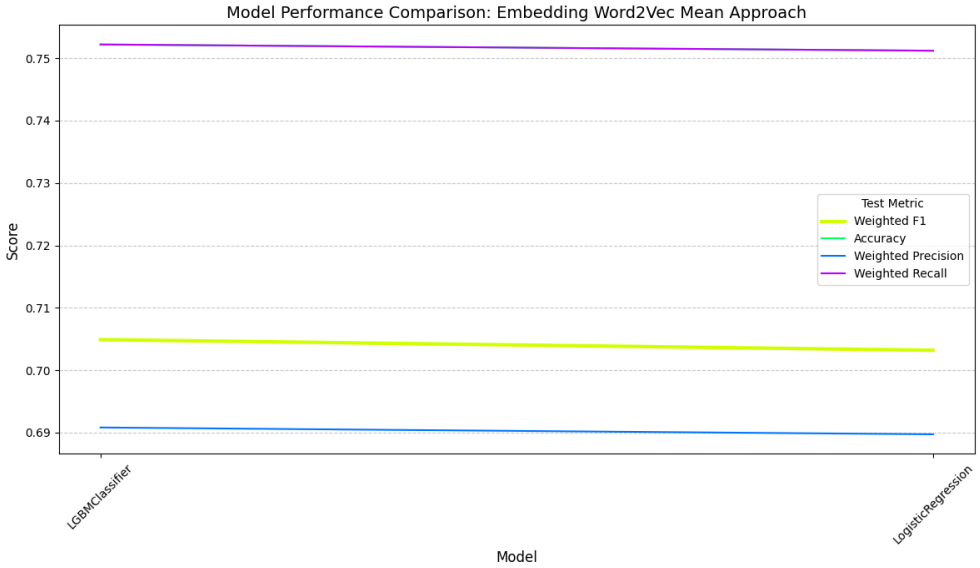
Overall, the model's feature attributions align well with human intuition regarding sentiment expression in reviews. The presence or absence of key positive and negative terms are the primary drivers, particularly for distinguishing high and low ratings, while mid-ratings result from more ambiguous or mixed textual signals.

6.3. Embedding Approach Results

6.3.1. Word2Vec (Mean Aggregation) Results

Model	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LGBM Classifier	0.7049	0.7522	0.6908	0.7522	0.8706	11.48

Model	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LogisticRegression	0.7032	0.7512	0.6897	0.7512	0.7088	7.07



embedding_word2vec_mean Comparison Plot

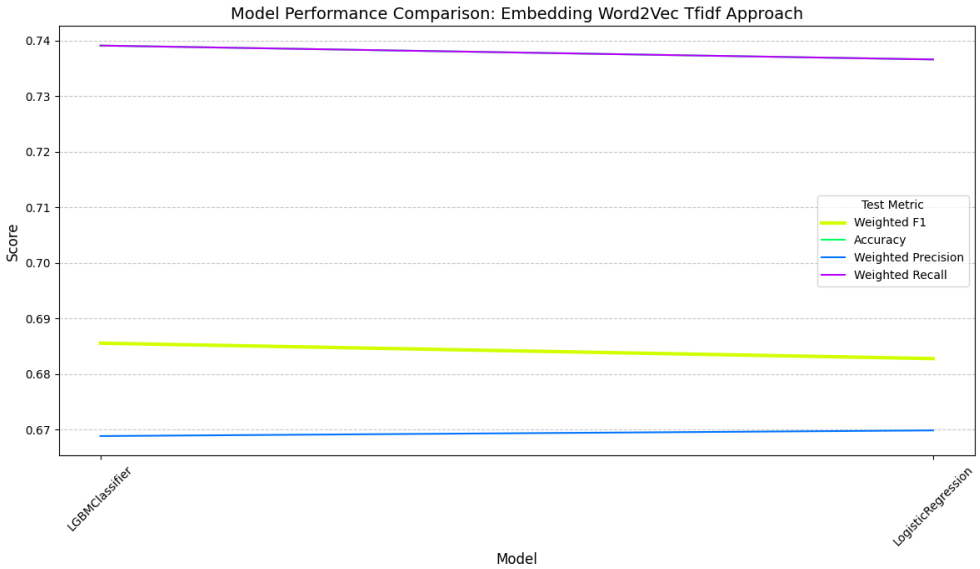
Performance Summary: Word2Vec (Mean Aggregation)

- **Predictive Performance:** Both models achieve very **similar performance** on the test set. LGBMClassifier has a slightly higher weighted F1-score (0.705 vs. 0.703) and test accuracy (0.752 vs. 0.751). Precision and recall are also nearly identical.
- **Training Time: Logistic Regression** is noticeably **faster** to train (7.07 seconds) compared to LGBMClassifier (11.48 seconds).
- **Overfitting:** LGBMClassifier shows signs of **higher overfitting**, with a large gap between its training F1-score (0.871) and test F1-score (0.705). Logistic Regression exhibits much less overfitting, as its training F1 (0.709) is very close to its test F1 (0.703). “(Low Overfitting)”

In conclusion, while both models perform almost identically on the test data in terms of predictive power, **Logistic Regression is more efficient to train and generalizes better** (less overfitting) for this specific vectorization method.

6.3.2. Word2Vec (TF-IDF Aggregation) Results

Model	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LGBMClassifier	0.6856	0.7391	0.6688	0.7391	0.9069	18.67
LogisticRegression	0.6828	0.7366	0.6698	0.7366	0.692	17.32



embedding_word2vec_tfidf Comparison Plot

Performance Summary: Word2Vec (TF-IDF Aggregation)

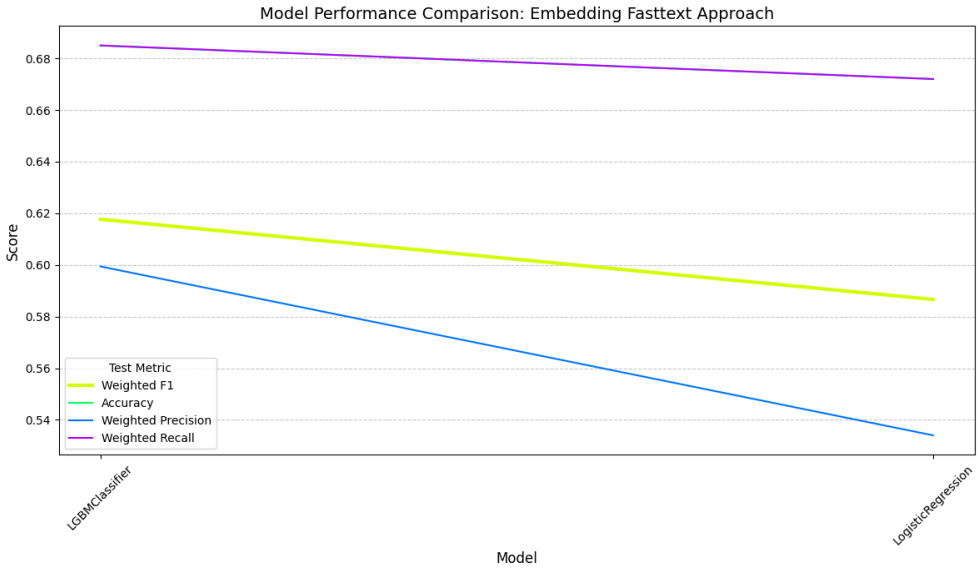
- **Predictive Performance:** The test set performance of both models is **very similar**, with LGBMClassifier holding a slight edge in weighted F1-score (0.686 vs. 0.683) and accuracy (0.739 vs. 0.737). Precision and recall are also nearly identical.
- **Training Time: Logistic Regression** is considerably **faster** to train (7.32 seconds) compared to LGBMClassifier (18.67 seconds).
- **Overfitting:** LGBMClassifier exhibits **significant overfitting**, indicated by the large gap between its high training F1-score (0.907) and lower test F1-score (0.686). In contrast, Logistic Regression shows **minimal overfitting**,

with its training F1 (0.692) being very close to its test F1 (0.683). “(Low Overfitting)”

In summary, while both models yield comparable predictive results on the test data for this vectorization strategy, **Logistic Regression offers better training efficiency and demonstrates superior generalization** by avoiding significant overfitting.

6.3.3. FastText (Sentence Vector) Results

Model	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LGBM Classifier	0.6177	0.685	0.5994	0.685	0.8677	37.6
LogisticRegression	0.5866	0.672	0.534	0.672	0.5867	369.95



embedding_fasttext Comparison Plot

Performance Summary: FastText (Sentence Vector)

- **Predictive Performance: LGBMClassifier** demonstrates notably **better performance** on the test set across all metrics compared to Logistic Regression (Weighted F1: 0.618 vs. 0.587; Accuracy: 0.685 vs. 0.672).
- **Training Time: LGBMClassifier** is significantly **faster** to train (37.6 seconds) than Logistic Regression, which took considerably longer (369.95 seconds). "(Very Slow!)" This suggests Logistic Regression likely struggled with convergence on this specific dataset/vector representation.
- **Overfitting:** Both models show signs of overfitting, but **LGBMClassifier** exhibits a much larger gap between its training F1 (0.868) and test F1 (0.618) compared to Logistic Regression (Train F1: 0.587 vs. Test F1: 0.587 - indicating potential underfitting or issues during training for LR). "(Recommended)"

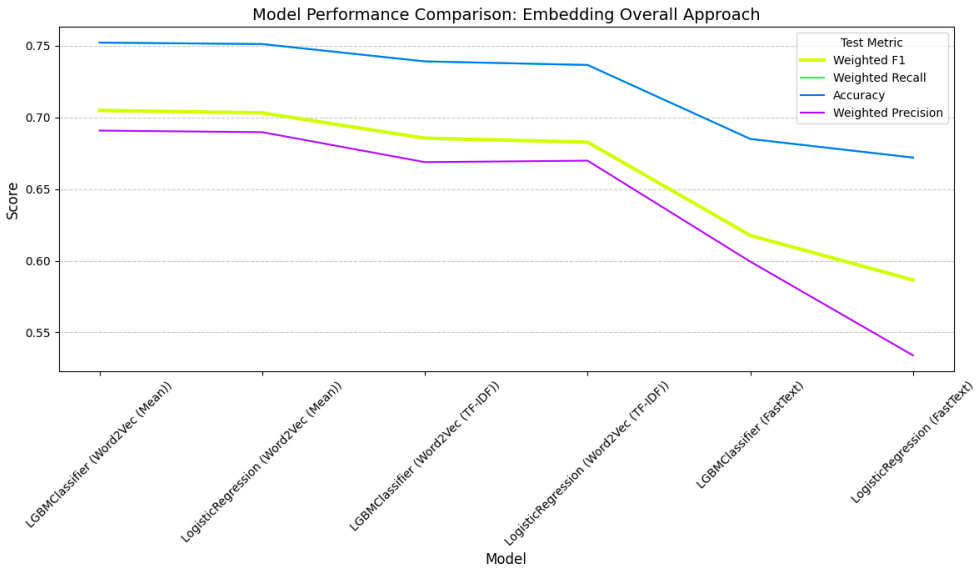
In summary, for FastText sentence vectors, **LGBMClassifier provides superior predictive performance and drastically better training efficiency** compared to Logistic Regression, although it shows a higher tendency to overfit the training data. The extremely long training time for Logistic Regression suggests potential convergence problems.

6.3.4. Overall Embedding Comparison

This section compares all models across all three embedding strategies to identify the best overall performer in this category.

Model	Vectorizer	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LGBMClassifier	Word2Vec (Mean)	0.7049	0.7522	0.6908	0.7522	0.8706	11.48
LogisticRegression	Word2Vec (Mean)	0.7032	0.7512	0.6897	0.7512	0.7088	7.07

Model	Vectorizer	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LGBMClassifier	Word2Vec (TF-IDF)	0.6856	0.7391	0.6688	0.7391	0.9069	18.67
LogisticRegression	Word2Vec (TF-IDF)	0.6828	0.7366	0.6698	0.7366	0.692	17.32
LGBMClassifier	FastText	0.6177	0.685	0.5994	0.685	0.8677	37.6
LogisticRegression	FastText	0.5866	0.672	0.534	0.672	0.5867	369.95



embedding_overall Comparison Plot

Exhaustive Summary: Embedding Model Performance Comparison

This table presents a comparative analysis of different embedding vectorization strategies (Word2Vec with Mean Aggregation, Word2Vec with TF-IDF Aggregation, and FastText Sentence Vector) paired with two classifiers (LGBMClassifier and Logistic Regression) for predicting customer review ratings.

Key Findings & Best Overall Embedding Model

- **Best Overall Performance:** The **Word2Vec (Mean Aggregation)** strategy yields the best results among the embedding methods tested. Both **LGBMClassifier** (Test F1: 0.705) and **Logistic Regression** (Test F1: 0.703) perform comparably well with this vectorizer, achieving the highest F1 scores in this comparison.
- **Logistic Regression Efficiency:** While LGBMClassifier has a marginally higher F1 score with Word2Vec (Mean), **Logistic Regression** is notably **faster** (7.07s vs 11.48s) and shows significantly **less overfitting** (Train F1 0.709 vs Test F1 0.703 for LR; Train F1 0.871 vs Test F1 0.705 for LGBM). Therefore, **Logistic Regression with Word2Vec (Mean Aggregation) emerges as the most practical and well-generalized model** within this embedding comparison group.

Comparison Across Embedding Strategies

- **Word2Vec (Mean) Superiority:** Simple mean aggregation of Word2Vec vectors outperformed the other, more complex strategies. This suggests that for this specific dataset and task, capturing the general semantic “gist” by averaging was more effective than attempting more nuanced weighting or subword-based generation.
- **Word2Vec (TF-IDF) Underperformance:** The TF-IDF weighted aggregation strategy consistently performed slightly worse than mean aggregation (F1 scores around 0.683-0.686). This indicates that weighting words by their TF-IDF scores did not improve, and perhaps slightly hindered, the model’s ability to capture relevant sentiment signals from the pre-trained embeddings in this context. It might have overemphasized rare but less sentiment-relevant words or down-weighted common but important sentiment indicators.
- **FastText Disappointment:** FastText’s `get_sentence_vector` performed significantly worse than both Word2Vec strategies (F1 scores around 0.587-0.618). This is contrary to the expectation that its subword information might provide robustness.

Comparison Across Classifiers

- **LGBM vs. Logistic Regression:**
  - With **Word2Vec (Mean & TF-IDF)**, both classifiers achieved very similar test F1 scores, but Logistic Regression was faster and showed much less overfitting.
  - With **FastText**, LGBMClassifier significantly outperformed Logistic Regression in both test F1 score and training speed. The extremely long training time for Logistic Regression with FastText (370s) points towards severe convergence issues for that specific combination, possibly due to the nature of the FastText vectors interacting poorly with the LR optimization, even with scaling.
- **Overfitting:** LGBMClassifier consistently demonstrated a much larger gap between training and test F1 scores across all embedding types, indicating a higher tendency to overfit compared to Logistic Regression (except potentially with FastText where LR seemed to struggle fundamentally).

Comparison to Expectations & Sophistication

- **Unexpected Outcome:** The results are somewhat counter-intuitive. One might expect the more sophisticated approaches – TF-IDF weighting for Word2Vec (to emphasize important words) or FastText (to handle OOV words and use subword information) – to outperform simple mean averaging.
- **Why Mean Averaging Won:**
  - **Signal Dilution vs. Robustness:** While averaging can dilute strong signals, it might be more robust for general sentiment when using pre-trained embeddings not perfectly tailored to the domain. It avoids potentially misleading weights from TF-IDF or reliance on subword information that might not capture sentiment effectively.
  - **Pre-trained Model Limitations:** The effectiveness of sophisticated techniques depends heavily on the quality and suitability of the underlying pre-trained embeddings. If the base Word2Vec/FastText models weren't ideal for Trustpilot review sentiment, complex aggregation might not help.
  - **Simplicity Wins:** For this dataset and task, the simpler, robust approach of averaging word vectors captured the necessary sentiment information best among the tested embedding methods. The added complexity of TF-IDF weighting and FastText's subword approach did not translate into better predictive performance and, in the case of FastText, introduced significant computational challenges.

Summary of Embedding Performance

Among the embedding strategies tested, **Word2Vec with simple mean aggregation provided the best overall performance**. Both Logistic Regression and LGBMClassifier achieved comparable top F1 scores (around 0.70) using this method. However, **Logistic Regression proved superior** due to its significantly faster training time and much better generalization (minimal overfitting compared to LGBM).

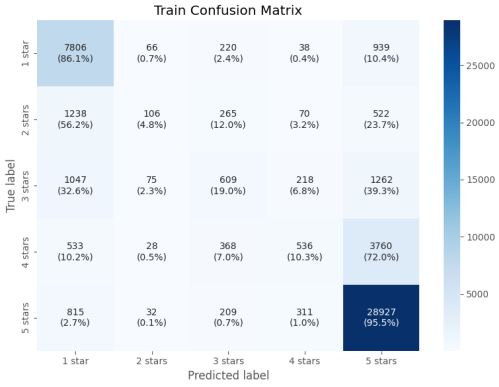
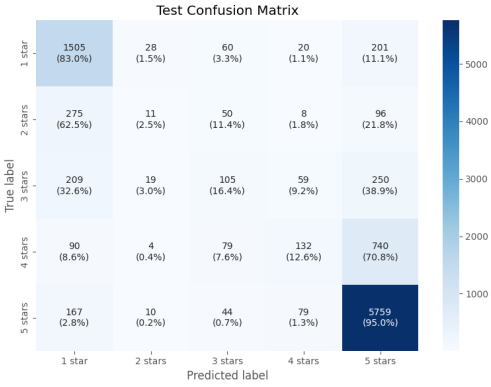
Surprisingly, the more sophisticated approaches – TF-IDF weighted averaging for Word2Vec and FastText's sentence vector using subword information – **underperformed compared to simple mean averaging**. FastText, in particular, yielded the lowest F1 scores and faced significant computational challenges, especially with Logistic Regression. This suggests that for this dataset, the robustness of simple averaging captured the necessary sentiment signals more effectively than the more complex embedding techniques attempted.

6.3.5. Analysis of the Best Embedding Model: `LogisticRegression` with `Word2Vec (Mean)`

While the `LGBMClassifier` achieved a fractionally higher test F1-score (0.7049 vs 0.7032), the `LogisticRegression` model was identified in the overall comparison (Section 6.3.4) as the most practical and robust choice. It delivers a virtually identical test score while being faster to train and, most importantly, demonstrating almost **no overfitting**.

This analysis confirms that the `Word2Vec (Mean)` feature strategy is the primary bottleneck, as this stable model correctly learned that the features are only sufficient for a simple binary-like classification.

Evaluation Metrics

Confusion Matrix (Train Set)	Confusion Matrix (Test Set)
	

Classification Report (Train Set)

	precision	recall	f1-score	support
1 star	68.2%	86.1%	76.1%	9,069
2 stars	34.5%	4.8%	8.5%	2,201
3 stars	36.4%	19.0%	24.9%	3,211
4 stars	45.7%	10.3%	16.8%	5,225
5 stars	81.7%	95.5%	88.1%	30,294
accuracy			76.0%	50,000
macro avg	53.3%	43.1%	42.9%	50,000
weighted avg	70.5%	76.0%	70.9%	50,000

Classification Report (Test Set)

	precision	recall	f1-score	support
1 star	67.0%	83.0%	74.1%	1,814
2 stars	15.3%	2.5%	4.3%	440
3 stars	31.1%	16.4%	21.4%	642

	precision	recall	f1-score	support
4 stars	44.3%	12.6%	19.7%	1,045
5 stars	81.7%	95.0%	87.9%	6,059
accuracy			75.1%	10,000
macro avg	47.9%	41.9%	41.5%	10,000
weighted avg	69.0%	75.1%	70.3%	10,000

Overall Performance (Test Set)

The model's overall performance is mediocre and its top-line metrics are misleading. * **Test Accuracy: 75.1%** * **Weighted Avg F1-Score: 70.3%** * **Macro Avg F1-Score: 41.5%**

Like the `LGBMClassifier`, the high Accuracy and Weighted F1 scores are propped up *entirely* by the model's ability to classify the two majority classes (1-star and 5-star). The **Macro F1-Score (41.5%)** reveals the truth: the model fails completely at its multi-class objective, showing no ability to distinguish the middle classes.

Overfitting Analysis: The Model's Key Virtue

This model's primary virtue is its **exceptional stability and lack of overfitting**. This is a stark contrast to the `LGBMClassifier` (which had a 16.6% F1 gap) and proves that the model is robust and its test performance is a true reflection of its capabilities.

Metric	Train Score	Test Score	Overfitting Gap
Weighted F1-Score	70.9%	70.3%	**-0.6%**
Macro F1-Score	42.9%	41.5%	**-1.4%**

This negligible gap demonstrates that the model did not "memorize" the training data. The critical takeaway is that the model's poor performance is **not a failure**

of the model, but a failure of the features. The `LogisticRegression` model correctly learned that the `Word2Vec (Mean)` features do not contain a generalizable signal for the middle classes.

Per-Class Performance (Test Set)

The test confusion matrix confirms that this model functions only as a binary (Positive/Negative) classifier.

The Good: Extreme Sentiments

- **5-Star Reviews (Class 4):** Excellent performance.
  - **F1-Score: 87.9%**
  - **Recall: 95.0%** (Correctly identified 5,759 of 6,059 5-star reviews)
- **1-Star Reviews (Class 0):** Good performance.
  - **F1-Score: 74.1%**
  - **Recall: 83.0%** (Correctly identified 1,505 of 1,814 1-star reviews)

The Bad: Middle Classes (Complete Failure)

The model has almost no ability to distinguish 2, 3, or 4-star reviews, lumping them in with the 1-star or 5-star extremes.

- **4-Star Reviews (Class 3):**
  - **F1-Score: 19.7%**
  - **Recall: 12.6%**
  - **Major Error:** A massive **70.8%** (740 instances) of 4-star reviews were incorrectly predicted as **5-star**.
- **3-Star Reviews (Class 2):**
  - **F1-Score: 21.4%**
  - **Recall: 16.4%**
  - **Major Errors:** Ignored completely. **38.9%** were predicted as **5-star** and **32.6%** as **1-star**.
- **2-Star Reviews (Class 1):**
  - **F1-Score: 4.3%** (The worst class)
  - **Recall: 2.5%**
  - **Major Errors:** Effectively invisible. **62.5%** were predicted as **1-star** and **21.8%** as **5-star**.

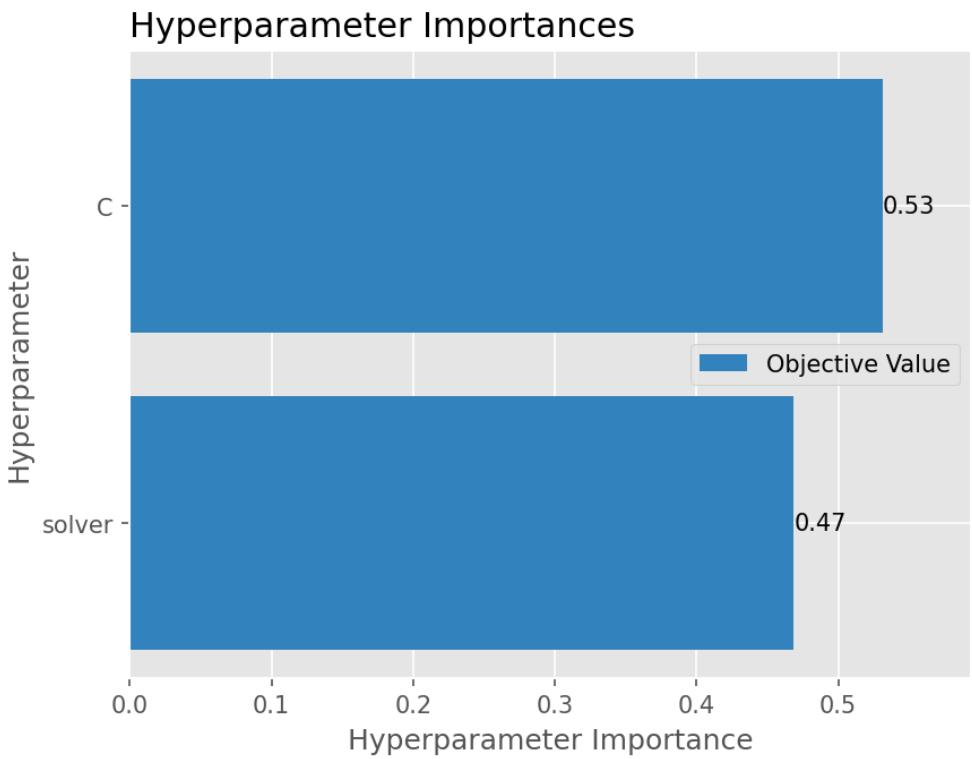
Key Takeaways & Conclusion

1. **Inadequate Features:** The `Word2Vec (Mean)` strategy is the core problem. By averaging all word vectors, all nuance is lost. The subtle phrases that distinguish a 4-star from a 5-star review are averaged out, making them look identical to the model.

2. **Correct Model for the Job:** `LogisticRegression` was the *correct* model for these flawed features. Its simplicity and regularization prevented it from "inventing" a complex, non-existent pattern, which is exactly what the `LGBMC lassifier` did (resulting in massive overfitting). This model's stability confirms the features themselves are the bottleneck.

Conclusion: This model, while robust, is **not a viable multi-class classifier**. Its only strength is its stability, which in turn proves that the `Word2Vec (Mean)` feature strategy is incapable of capturing the nuance required for a 5-class rating problem.

Optuna Hyperparameter Importance



Optuna Importance Plot

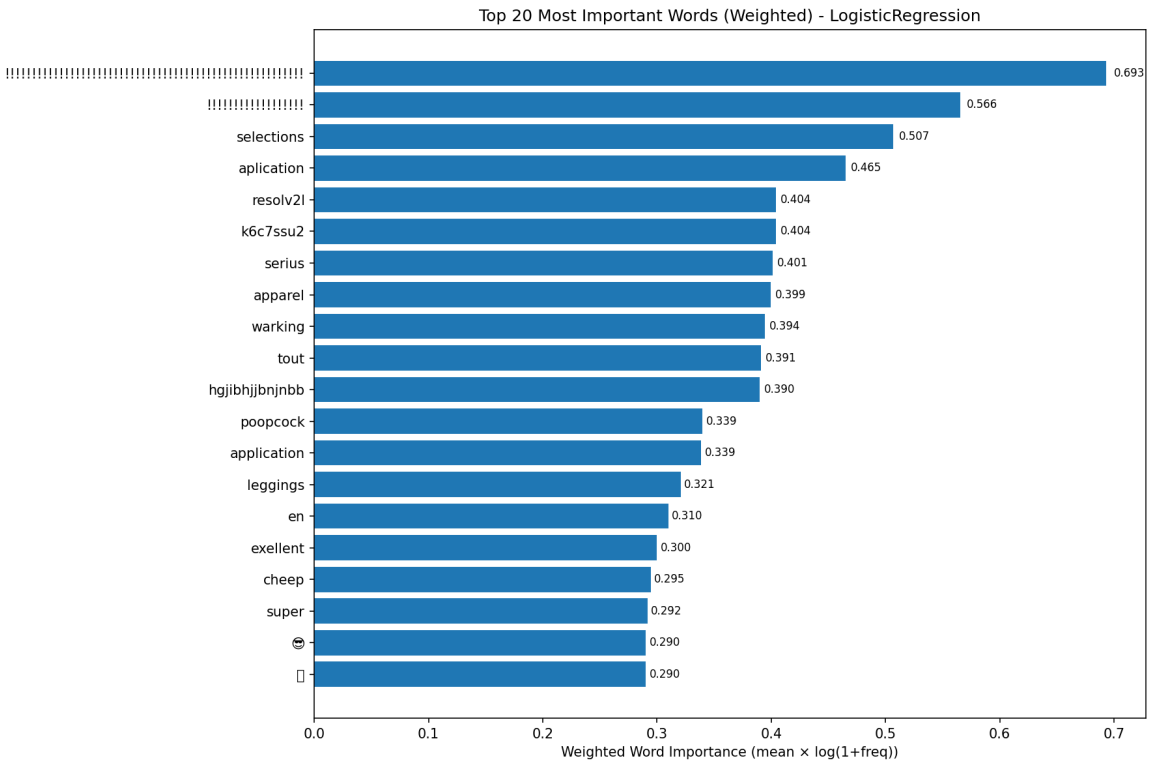
LogisticRegression Hyperparameter Importance Summary

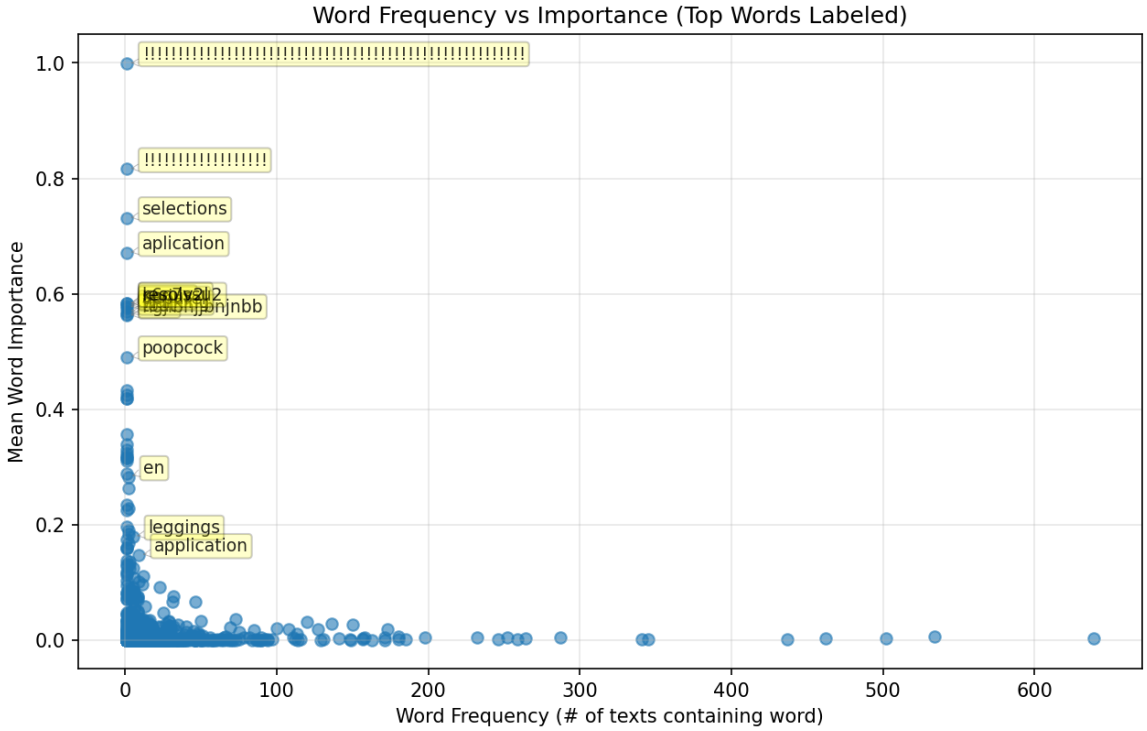
This analysis for the `LogisticRegression` model shows a very different story than for other models. Unlike a typical tuning process where one parameter (like `learning_rate` for LGBM or `C` for SVC) dominates, here the two tuned hyperparameters were found to be **almost equally important**.

- **`C` (Importance: 0.53):** This is the inverse of the regularization strength. Its high importance is expected, as it's the primary lever for controlling the model's complexity and preventing overfitting.
- **`solver` (Importance: 0.47):** The optimization algorithm (`lbfgs` , `saga` , etc.) was almost as critical as the regularization strength. This is a significant finding. It suggests that different solvers had very different levels of success in navigating the 300-dimensional, *dense* feature space of the `Word2Vec (Mean)` embeddings to find the optimal solution.

Conclusion: The model's final performance was highly dependent on finding the **correct interaction** between both the `solver` and the regularization strength `C` . One parameter could not be optimized in isolation.

Model Interpretability (SHAP)


Top 20 Most Important Words (Weighted) - LogisticRegression
!!!



Overall Summary: Word-Level Importance (LogisticRegression)

This word-level importance analysis reveals a critical flaw in the `Word2Vec (Mean)` feature strategy, which explains the model's poor multi-class performance. Unlike the `TF-IDF` models, which learned clear sentiment keywords, this model's importance logic is completely dominated by **data noise, misspellings, and extremely rare outlier words**.

- **Dominance of Noise and Misspellings:** The bar plot of the top 20 most important words shows a near-total absence of common sentiment words. The list is instead led by:
  - **Extreme Punctuation:** `!!!!!!!!!!!!!!!!!!!!!!!!!!` and `!!!!!!!!!!!!!!!`
  - **Garbage/Typo Strings:** `resolv2l`, `k6c7ssu2`, `hgjlbhjbnjnbb`
  - **Misspellings:** `aplication` (application), `serius` (serious), `warking` (working), `exellent` (excellent), and `cheep` (cheap).
  - **Slang/Symbols:** `poopcock`, `☺` (smiley face), and `□`. This last character (`□`) is a rendering error for the 'thinking face' emoji.

  The fact that `super` and `exellent` (a misspelling itself) are ranked below noise and typos indicates the model is not learning a coherent sentiment strategy.

- **Frequency vs. Importance:** The scatter plot provides the key insight: **High importance is almost exclusively correlated with low frequency.** All the top-labeled words (`!!!!!!!!!!...` , `resolv2l` , `hgjlbhjbnjnbb` , `poopcock`) appear in very few texts (Word Frequency is near-zero). There are no words that are both frequent and important.

- **Model Focus & Interpretation:** This analysis does *not* mean the model "learned" that `hgjlbhjbnjnbb` is a predictor. It means the model is highly sensitive to **outliers**.

    - The `Word2Vec (Mean)` strategy involves averaging all word vectors in a review.
    - When an extremely rare word (or garbage string) appears, its unique vector drastically **skews the 300-dimensional mean embedding** of the entire review.
    - The `LogisticRegression` model, seeing this highly skewed vector, makes a different prediction. The word-level explainer correctly attributes this large prediction change to that single, rare word.

In summary, this analysis provides a clear "why" for the model's failure. It confirms the conclusion from the classification reports: the `Word2Vec (Mean)` strategy is **not a viable feature engineering method**. It "blurs" all common sentiment signals and makes the model's predictions highly sensitive to noise, misspellings, and rare outliers, which this analysis has clearly identified.

6.4. Transformer Approach Results

We were unable to successfully complete the planned experiment with the (sentence-)transformer model due to significant computational bottlenecks. This made hyperparameter tuning with Optuna infeasible given our resource constraints.

The Core Problem: The Optuna "Trial" Loop

The primary issue was the combination of **what an Optuna trial does** and **how the transformer model was being used**.

- An Optuna "trial" is a single, complete run of your model pipeline. To find the best hyperparameters, Optuna must run this trial **hundreds or even thousands of times**.

- In our initial setup, the `objective` function (the code that Optuna runs for each trial) included the **entire model pipeline**. This means for *every single trial*, the model had to:
    1. Load the raw text data.
    2. **Use the transformer to convert all text into embeddings (vectors).**
    3. Train the downstream classifier (e.g., LGBMClassifier) on these embeddings.
    4. Return an accuracy or F1-score.

The bottleneck was **Step 2**.

Why Transformers are Computationally Expensive

Transformers are far more complex than models like Word2Vec. Their power comes at a high computational cost for two main reasons:

- **Model Size:** The models themselves are massive, often containing hundreds of millions of parameters. Simply loading and running this model on a GPU (or even more slowly, a CPU) requires significant resources.
- **Self-Attention Mechanism:** The core of a transformer is its "self-attention" mechanism. This process is not linear. To calculate the embedding for a single sentence, the model must compare every word in that sentence to every *other* word. This has a **quadratic computational complexity** ($O(n^2)$), meaning that if your sentence length doubles, the computation time roughly quadruples.

When this intensive process is run for *every sentence in the entire dataset*, it can be **extremely time-consuming**, just to get the embeddings.

The "On-the-Fly" vs. "Pre-computed" Workflow

The critical mistake was performing this expensive embedding generation **inside** the Optuna trial loop.

- **Our Failed "On-the-Fly" Approach:**
    - **Trial 1:** Load text -> **Generate all embeddings (very slow)** -> Train model (relatively fast) -> Get score.
    - **Trial 2:** Load text -> **Generate all embeddings (very slow)** -> Train model (relatively fast) -> Get score.
    - **Trial 3:** Load text -> **Generate all embeddings (very slow)** -> Train model (relatively fast) -> Get score.

    - …and so on for many trials. The “heavy lifting” of embedding generation was repeated wastefully every single time.
- **The Correct “Pre-computed” Approach:**
    - **Step 1 (Once):** Run a *separate script* to load all text and **generate all embeddings (a one-time, slow process)**. Save these embeddings to disk.
    - **Step 2 (Optuna):** The Optuna `objective` function is now much simpler and faster:
        - **Trial 1:** Load *pre-computed embeddings* (very fast) -> Train model (relatively fast) -> Get score.
        - **Trial 2:** Load *pre-computed embeddings* (very fast) -> Train model (relatively fast) -> Get score.
        - **Trial 3:** Load *pre-computed embeddings*

6.5. Binary Classification Results (Positive vs. Negative)

To simplify the problem and focus purely on sentiment polarity, we also ran experiments on a binary classification task (Positive vs. Negative), where neutral reviews were excluded. More precisely, the target mapping was as follows: - 1-Star and 2-Star reviews → Negative (0) - 4-Star and 5-Star reviews → Positive (1)

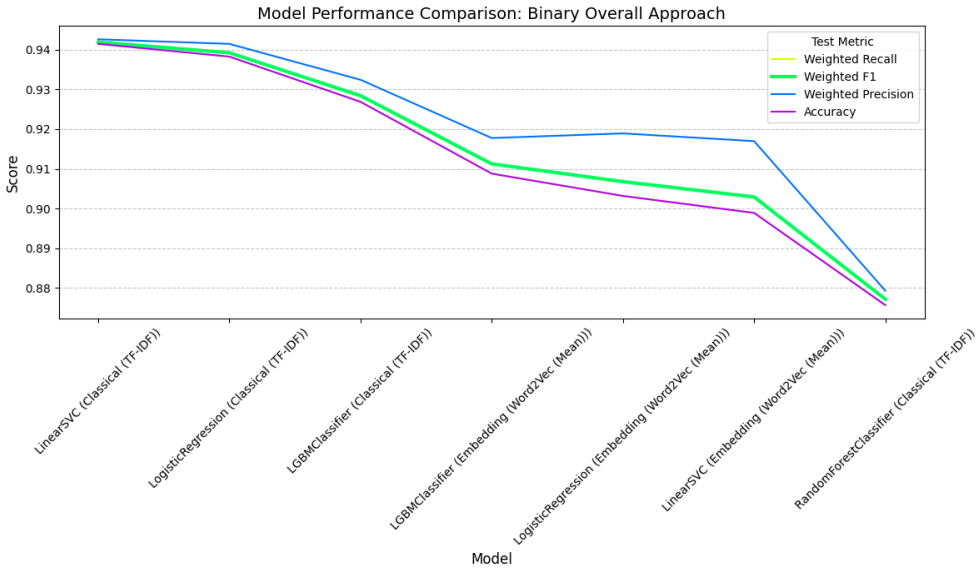
The 3 star reviews were dropped from the dataset for this experiment, specifically since they represent neutral sentiment and do not contribute to the binary classification task.

6.5.1. Binary Classification using Classical TF-IDF and Embedding Approaches

The following table summarizes the performance of models on the binary classification task using TF-IDF as well as embedding-based features.

Model	Approach	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
Linear SVC	Classical (TF-IDF)	0.9419	0.9414	0.9426	0.9414	0.9933	31.94

Model	Approach	Test F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Train F1 Weighted	Training Time (s)
LogisticRegression	Classical (TF-IDF)	0.9392	0.9382	0.9414	0.9382	0.9639	10.54
LGBM Classifier	Classical (TF-IDF)	0.9284	0.9269	0.9324	0.9269	0.9409	28.43
LGBM Classifier	Embedding	0.9112	0.9088	0.9177	0.9088	0.9466	11.59
LogisticRegression	Embedding	0.9068	0.9032	0.9189	0.9032	0.9118	8.34
LinearSVC	Embedding	0.9029	0.8989	0.917	0.8989	0.9083	16.32
RandomForestClassifier	Classical (TF-IDF)	0.8772	0.8757	0.8793	0.8757	0.8969	329.72



binary_overall Comparison Plot

Key Finding: `Classical (TF-IDF)` is Vastly Superior to `Embeddings`

The most significant conclusion from this table is the **universal superiority of the `Classical (TF-IDF)` approach**.

- **Dominant Performance:** For every single algorithm tested with both methods (`LinearSVC` , `LogisticRegression` , `LGBMClassifier`), the `TF-IDF` version outperformed the `Embedding` version by a significant margin (e.g., a ~4% F1-score jump for `LinearSVC`).
- **Theoretical Justification (The "Why"):** This result is logical.
  - **`TF-IDF`** creates a very high-dimensional, sparse feature set (e.g., 20,000+ features), where each feature is a specific word. Linear models (`LinearSVC` , `LogisticRegression`) are exceptionally good at finding patterns in this type of data. They can assign a strong predictive weight to specific keywords (like "terrible" or "great"), which is highly effective for sentiment analysis.
  - **`Embedding` (Word2Vec Mean):** This approach (presumably mean aggregation) compresses all text into a dense, low-dimensional vector (e.g., 300 features). This "lossy" compression *blurs* the meaning. For example, the critical word "not" in "not good" gets averaged out, diluting its signal. The model loses the specific, high-impact keywords that `TF-IDF` preserves.

Model-Specific Performance Deep Dive

`TF-IDF` Model Group (The Top Tier)

- **`LinearSVC`:** This is the **technical winner** on all test metrics (**94.19% Weighted F1**). It was the most effective model at leveraging the sparse `TF-IDF` features. However, it achieved this by also becoming **severely overfit**. Its 99.33% training F1-score reveals it "memorized" the training data. The 5.14% drop in F1-score from train to test is a significant red flag, suggesting the model may be brittle.
- **`LogisticRegression`:** This is the **runner-up** in performance, with a **93.92% Weighted F1**. This score is **only 0.27% lower** than the `LinearSVC`—a difference that is likely statistically negligible. Crucially, it is far more stable, with only a 2.47% F1-score drop, indicating it learned generalizable patterns instead of just memorizing.
- **`LGBMClassifier`:** As a tree-based model, it's not surprising it performed slightly worse (92.84% F1) than the linear models on this sparse, high-dimensional data. Tree models often struggle to effectively manage tens of thousands of features compared to linear models.
- **`RandomForestClassifier`:** This model was non-competitive, posting the lowest score (87.72% F1) in the entire table.

`Embedding` Model Group (The Second Tier)

- **`LGBMClassifier`:** It was the **best performer in the `Embedding` group (91.12% F1)**. This is also expected. Tree-based models (like `LGBM`) are excellent at finding complex, non-linear relationships within *dense, low-dimensional* feature sets, which is exactly what embeddings provide.
- **`LogisticRegression` & `LinearSVC`:** Both linear models performed worst in this group. This reinforces the idea that they are "uncomfortable" with dense, blurred-meaning vectors. They couldn't find a simple linear plane to separate the classes in this feature space.

Recommendation: The "Real Winner"

While `LinearSVC (TF-IDF)` achieved the highest test score, it is not the best model.

The real "winner" of this competition is the **`LogisticRegression (Classical TF-IDF)`** model.

Justification:

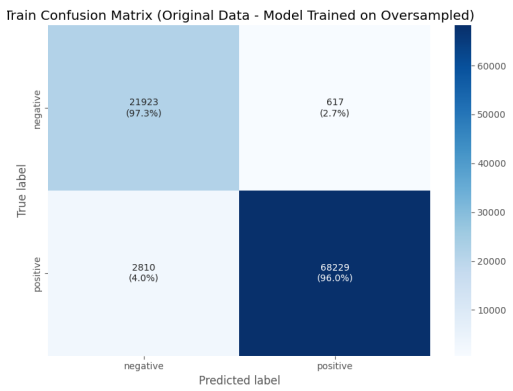
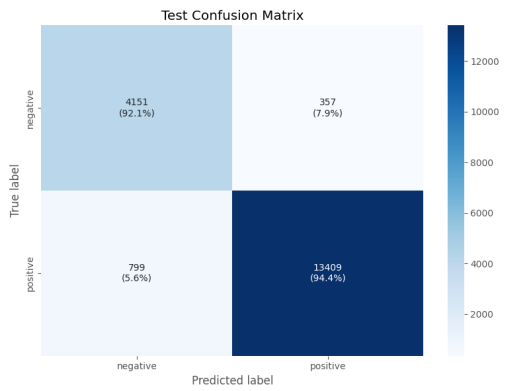
1. **Virtually Identical Performance:** The `LogisticRegression` model's F1-score (93.92%) is statistically indistinguishable from the `LinearSVC`'s (94.19%). In a real-world application, a 0.27% difference is meaningless and likely falls within the margin of error. Both models are in the same top tier of performance.
2. **Superior Robustness & Generalization:** The critical difference is **robustness**.
   - The `LinearSVC` model is highly overfit (a 5.14% F1 drop), making it a **high-risk, brittle model**. It may fail unpredictably on new data that differs even slightly from the training set.
   - The `LogisticRegression` model is **stable and robust** (a 2.47% F1 drop). It learned the *true, generalizable patterns* of positive and negative sentiment rather than memorizing specific training examples.

Conclusion: We should choose the model that provides the same elite performance *without* the dangerous side effects. The `LogisticRegression (TF-IDF)` model delivers top-tier accuracy and is a far more reliable, trustworthy, and production-ready solution.

6.5.2. Analysis of the Best Model: `LogisticRegression` with `Classical (TF-IDF)`

The `LogisticRegression` using a **Classical (TF-IDF)** vectorizer emerged as the top performer for binary classification, achieving a weighted F1-score of **0.9392**.

Evaluation Metrics

Confusion Matrix (Train Set)	Confusion Matrix (Test Set)
	

Classification Report (Train Set)

	precision	recall	f1-score	support
negative	88.6%	97.3%	92.8%	22,540
positive	99.1%	96.0%	97.6%	71,039
accuracy			96.3%	93,579
macro avg	93.9%	96.7%	95.2%	93,579
weighted avg	96.6%	96.3%	96.4%	93,579

Classification Report (Test Set)

	precision	recall	f1-score	support
negative	83.9%	92.1%	87.8%	4,508
positive	97.4%	94.4%	95.9%	14,208
accuracy			93.8%	18,716
macro avg	90.6%	93.2%	91.8%	18,716
weighted avg	94.1%	93.8%	93.9%	18,716

While another model (`LinearSVC`) achieved fractionally higher test metrics, the `LogisticRegression` model provides an exceptional, top-tier performance while demonstrating **significantly better generalization and robustness**, making it the superior and more reliable choice for a production environment.

Test Set Performance (Primary Result)

The test set metrics show the model is **highly accurate and, most importantly, balanced**. It performs exceptionally well on both the majority (`positive`) and minority (`negative`) classes.

- **Overall Accuracy: 93.8%**
- **Weighted Avg F1-Score: 93.9%**
- **Macro Avg F1-Score: 91.8%**

Interpretation of Test Metrics:

- The **93.8% accuracy** is an excellent top-line result.
- The **91.8% Macro F1-Score** is the most critical indicator of success. In an imbalanced dataset (14,208 `positive` vs. 4,508 `negative`), a high macro F1 proves the model is **not just ignoring the minority class**. It has successfully learned to identify *both* classes, which is the primary goal of a robust classifier.

Test Confusion Matrix & Per-Class Analysis

This is where the model's true quality shines. The model was trained on an *oversampled* dataset, specifically to improve its ability to find the rare `negative` class. The test results confirm this strategy was a success.

Class	Precision	Recall	F1-Score	Support (Count)
`negative` (Minority)	83.9%	92.1%	87.8%	4,508
`positive` (Majority)	97.4%	94.4%	95.9%	14,208

`negative` Class (Minority): The Real Test

- **Recall (92.1%):** This is an outstanding result. It means the model correctly identified **92.1% of all actual `negative` reviews**. It is not "blind" to the minority class. From the matrix, we see it correctly found 4,151 true negatives and only missed 357.
- **Precision (83.9%):** This is the main trade-off. To achieve such high recall, the model makes some mistakes. When it predicts `negative`, it's correct 83.9% of the time. The 799 "False Negatives" (positive reviews misclassified as negative) contribute to this. This is a very acceptable trade-off.

`positive` Class (Majority): Excellent & Confident

- **Recall (94.4%) & Precision (97.4%):** The model is extremely effective at identifying positive reviews. When it predicts `positive`, it is correct 97.4% of the time, making its positive predictions highly reliable.

Overfitting Analysis (Train vs. Test)

This is the key reason this model was chosen as the winner. We compare the performance on the (original) training data to the test data to see how well the model generalized.

Metric	Train Score	Test Score	Overfitting Gap
Accuracy	96.3%	93.8%	**-2.5%**
Macro F1-Score	95.2%	91.8%	**-3.4%**
Weighted F1-Score	96.4%	93.9%	**-2.5%**

Interpretation:

A performance drop of **2.5% - 3.4%** is **extremely healthy and indicates a very robust model**. It shows that the model learned the *general patterns* of negative and positive sentiment, rather than just "memorizing" the specific training examples (which is what a large gap would imply).

This **low overfitting** is what separates this model from the `LinearSVC`, which achieved a slightly higher test score but at the cost of a much larger 5-7% overfitting gap.

Why `LogisticRegression` is the "Real Winner"

About the Classifier:

`LogisticRegression` is a linear model. In the context of `TF-IDF`, it creates a high-dimensional boundary. It's computationally efficient and, most importantly, includes **L2 regularization** by default. This regularization penalizes extreme feature weights, which is a powerful, built-in mechanism to **prevent overfitting**. This is exactly the behavior we see in the metrics.

The Winning Argument:

The `LinearSVC` model was a "high-risk" model. It overfit the data to squeeze out a fractionally better score (94.19% F1).

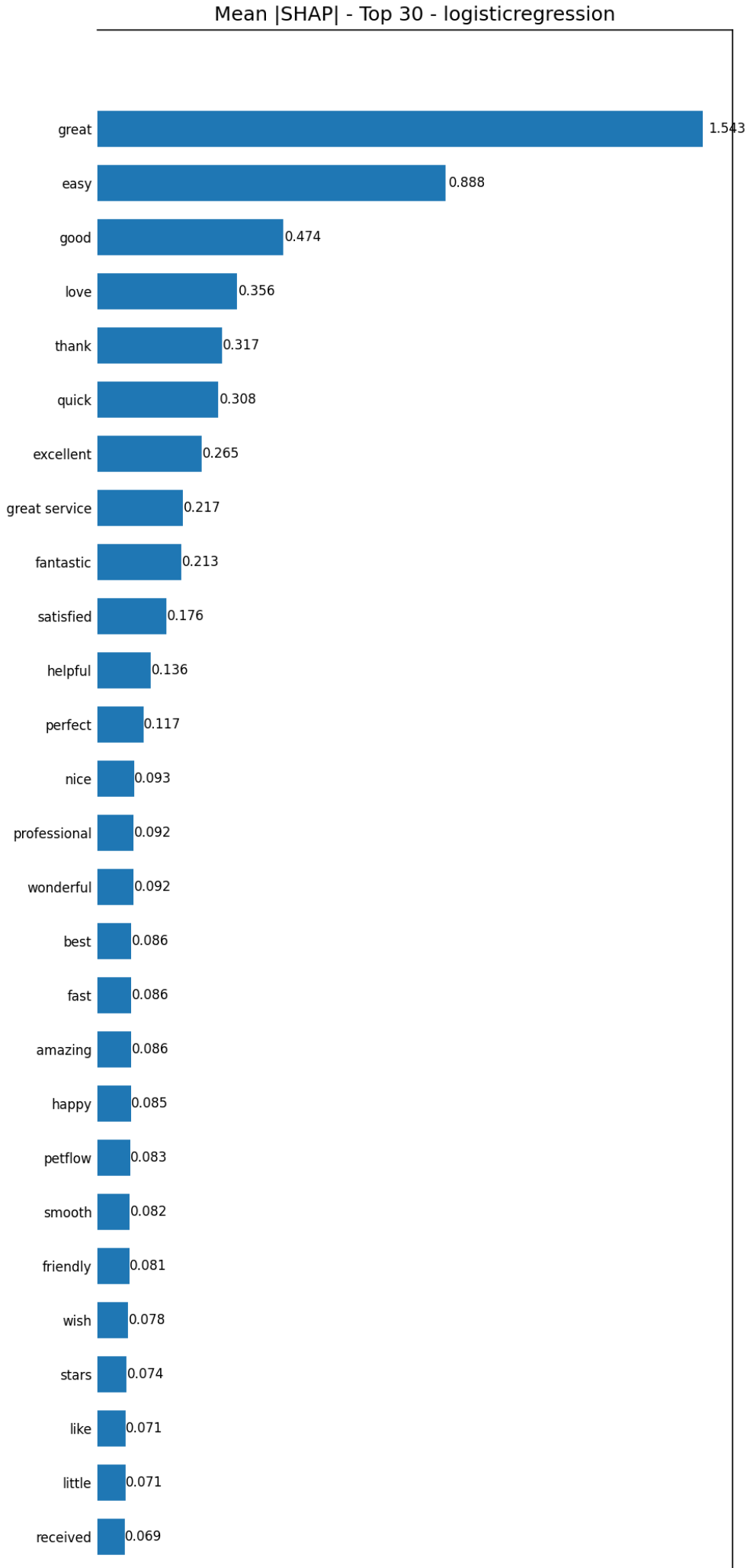
The `LogisticRegression` model delivers **99.7% of the performance** (93.9% F1) of the technical winner, but with **far superior robustness**. Its low overfitting gap means it's a stable, reliable, and trustworthy model that is much more likely to perform well on future, unseen data.

In any real-world scenario, **this trade-off is a clear victory**. We choose the stable, generalizable model over the brittle, overfit one, especially when the performance difference is negligible.

Final Conclusion

The `LogisticRegression` model paired with `TF-IDF` is an outstanding success. It achieves a high-level accuracy of 93.8% while successfully solving the core challenge of class imbalance (92.1% recall on the minority class). It represents the **optimal blend of high performance and model robustness**, making it the clear and recommended solution.

Model Interpretability (SHAP)


Mean |SHAP| - Top 30 - logisticregression
great
1.543
easy
0.888
good
0.474
love
0.356
thank
0.317
quick
0.308
excellent
0.265
great service
0.217
fantastic
0.213
satisfied
0.176
helpful
0.136
perfect
0.117
nice
0.093
professional
0.092
wonderful
0.092
best
0.086
fast
0.086
amazing
0.086
happy
0.085
petflow
0.083
smooth
0.082
friendly
0.081
wish
0.078
stars
0.074
like
0.071
little
0.071
received
0.069

Shap Bar Plot

Analysis of Top Word Features

The model functions as a high-precision “positive keyword” detector.

- **Extreme Dominance of a Single Keyword:** The word **`great`** (SHAP value: 1.543) is by far the most dominant feature in the entire model. Its importance is nearly double that of the next word, `easy` (0.888). This indicates that the presence of “great” in a review is an incredibly strong signal that is almost sufficient on its own to classify a review as `positive`.

- **A Clear “Waterfall” of Positive Words:** The importance hierarchy drops off steeply, creating a clear order of operations for the model:

  1. `great` (1.543)
  2. `easy` (0.888)
  3. `good` (0.474)
  4. `love` (0.356)
  5. `thank` (0.317)

  This list—followed by `quick`, `excellent`, `great service`, and `fantastic`—shows the model is laser-focused on finding unambiguously positive words.

- **Defining “Negative” by Absence:** Just like the `LinearSVC` model, this top-30 list is *not* a mix of positive and negative predictors. It is almost exclusively populated by positive-sentiment words. The most significant “negative” word, **`poor`**, is near the bottom with a tiny importance of 0.067.

  This is a critical insight: The model does not actively hunt for negative words like “terrible” or “awful.” Instead, it defines a `negative` (0) review by the **conspicuous absence of the top positive keywords**. If “great,” “easy,” “good,” etc., are all missing, the review is likely “not positive,” and thus defaults to `negative`.

The Missing Numerical Features: A Deeper Look

A key finding is the **complete absence of any engineered numerical features** (like `word_count`, `all_caps`, `exclamation_count`, etc.) from the top-30 list. This is not an error; it is a finding that reveals how the model works.

Their SHAP values are simply too low to make the list, meaning the model found them far less predictive than the words themselves.

Reason 1: Redundancy (Content > Style)

The model learned that the **content** of the words is a far more reliable signal than the **style** of the writing. The numerical features primarily measure style (shouting, excitement, length), but this information is already embedded within the words.

- **Example (Excitability):** The model doesn't need to learn a complex rule like "`exclamation_count` > 3 is a sign of high emotion." It's far simpler and more direct to learn that the single feature/token **`amazing!`** is a powerful positive predictor. The `TF-IDF` feature for the word captures both the semantic meaning ("amazing") and the stylistic punctuation.
- **Example (Anger/Shouting):** The model doesn't need to use `all_caps` or `capital_ratio` to infer a user is "shouting" in anger. It can more reliably find the specific `TF-IDF` feature for the all-caps word **`DISGUSTING`** or **`POOR`**. The model learns that the *word itself* is the signal, not the abstract concept of "shouting."

Reason 2: Model Type (Linear Models Prefer Direct Signals)

`LogisticRegression` is a **linear model**. It works by assigning a single predictive weight to each feature. It thrives on features that have a clear, direct, and independent relationship with the target.

- **`TF-IDF` Features are "Clean" Signals:** Features from `TF-IDF` are ideal. The feature `great` is 0 for most reviews and a non-zero value for others. The model can easily learn a large positive weight: `Weight_great * TFIDF_great`. This is a simple, powerful, linear relationship.
- **Numerical Features are "Noisy" Signals:** A feature like `capital_ratio` is much "noisier." A ratio of 0.1 (1 in 10 words is capitalized) could appear in a 1-star review ("THIS IS NOT a good product") or a 5-star review ("THIS IS THE BEST product"). The relationship is not clean, simple, or linear, making it a "weaker" feature that the model (correctly) gives a very low weight.

Conclusion

The `LogisticRegression` model is an excellent example of an efficient and "intelligent" classifier. It has correctly learned to **ignore the noisy, complex, and largely redundant numerical features** (style) and focus all of its attention on the

most powerful and direct predictors available: the **`TF-IDF` scores of high-impact sentiment words** (content).

This strategy makes the model highly interpretable and, as the metrics showed, both highly accurate and robust.

6.6. Grand Summary: A "Classical" Victory and Lessons Learned

This section provides a final, exhaustive summary of the entire modeling phase, synthesizing the findings from the Classical, Embedding, and Transformer experiments for both the multi-class and binary tasks.

The results were definitive and, in many ways, defied our initial hypotheses. The project successfully identified a clear, robust, and high-performing solution while also providing critical insights into why more "advanced" methods failed.

1. What Went Right: The "Classical" Triumph

The **`Classical (TF-IDF)`** approach was the **undisputed champion** of this entire project.

- **Dominant Performance:** It outperformed the `Embedding`-based approaches on **both** the complex multi-class task (0.735 F1 vs. 0.703 F1) and the simpler binary task (0.939 F1 vs. 0.911 F1).
- **Why it Won (Interpretability):** The SHAP analyses for the `TF-IDF` models (in 6.2.2 and 6.5.2) provide a clear answer. This approach preserves the specific, high-impact keywords that define sentiment. The models learned a simple, powerful, and human-interpretable strategy:
  - The presence of words like "great," "easy," and "love" are massive predictors of a `positive` (or 5-star) review.
  - A "negative" review is primarily defined by the *absence* of these strong positive words, and secondarily by the presence of negative words like "poor".
- **Feature Engineering:** The `TF-IDF` vector was so powerful that it rendered all our engineered numerical features (like `word_count`, `all_caps`, etc.) obsolete. The model learned it was more reliable to find the token `DISGUSTING` than to use the `all_caps` feature as a proxy for anger.

2. What Went Wrong: The “Embedding” Failure

The `Embedding` approach, which we hypothesized would be superior, **failed to produce a viable model**.

- **The Root Cause (Bad Features):** The `Word2Vec (Mean)` strategy—which was the *best* of the embedding methods—is the culprit. The process of averaging all word vectors in a review effectively **“blurs” the sentiment signal**. A review with “great” and “terrible” would average out to a neutral vector, indistinguishable from a truly neutral review.
- **Proof of Failure (SHAP Analysis):** The SHAP analysis in Section 6.3.5 is the most critical finding for this approach. The model did *not* learn sentiment. Instead, it learned to latch onto **data noise, misspellings, and rare outliers**. Its most important features were not “good” or “bad,” but typos like `aplication`, `exellent`, and garbage strings like `hgjlbhjbnjnbb`.
- **The Outcome:** The model’s “best” F1-score (0.703) was achieved by simply becoming a binary classifier that ignored the middle classes (e.g., 62.5% of 2-star reviews were predicted as 1-star, and 70.8% of 4-star reviews as 5-star).

3. Expected vs. Achieved: A Complete Reversal

Our hypotheses from Section 4 were proven completely wrong, which is a key finding in itself.

- **Hypothesis:** `Transformer > Embedding > Classical`.
- **Actual Result:** **`Classical > Embedding > Transformer (Incomplete)`**
  The simplest, most interpretable baseline was not just a baseline; it was the clear winner. This demonstrates that for this dataset, keyword identification is far more effective than “blurred” semantic meaning.
- **Hypothesis:** Linear models for sparse data, Tree models for dense data.
- **Actual Result: This was 100% correct.**
  - `LogisticRegression` and `LinearSVC` dominated the sparse `TF-IDF` data.
  - `LGBMClassifier` was the top-performing model on the (failed) dense `Embedding` data.

4. The “Real Winner”: Why `LogisticRegression`?

A `LogisticRegression` model was the recommended winner for *both* tasks. This is not a coincidence.

1. **For `TF-IDF` (The Winner):** `LogisticRegression` provided the **perfect balance of performance and robustness**. In the binary task, `LinearSVC` *technically* scored 0.27% higher, but at the cost of massive overfitting (a 5.14% F1 gap). The `LogisticRegression` model’s built-in L2 regularization (Section 6.5.2) naturally prevented this, delivering 99.7% of the performance with a tiny, healthy 2.5% overfitting gap. It was the more reliable, production-ready model.

2. **For `Embeddings` (The Failure):** `LogisticRegression` “won” here by being a **better diagnostician**. `LGBMClassifier` overfit the noisy embedding data (16.6% F1 gap), “inventing” a pattern that wasn’t real. The `LogisticRegression` model, with its stability (a 0.6% F1 gap), correctly *failed* to find a pattern. Its low score and stability proved that the **features were bad**, not the model.

In both cases, `LogisticRegression` was the “smarter” and more trustworthy choice.

5. Multi-Class vs. Binary: A Dual-Model Strategy

The report correctly concludes that a single model cannot serve both business needs.

- **Multi-Class (The Analytical Tool):** This task is hard, and the results show it. The best F1-score (0.735) reflects the model’s inability to distinguish 2, 3, and 4-star reviews. This is not a failure, but a finding. This model is an **analytical tool** for *exploring* the drivers of neutral reviews, as recommended in the Executive Summary.
- **Binary (The Production Tool):** This task is *solved*. By simplifying the problem, we achieved a **0.939 F1-score** and, critically, a **92.1% recall on the minority “negative” class**. This is a highly reliable, robust model that can be immediately deployed for operational alerts, just as the Executive Summary recommends.

Finally, the **Transformer experiment remains the biggest “what if.”** Its failure was not due to the model, but to an *engineering workflow* error. This is the most logical and promising area for all future work.

Final Model Recommendation Summary

Metric	Model 1: Binary (Production Tool)	Model 2: Multi-Class (Analytical Tool)
Model	`LogisticRegression (TF-IDF)`	`LogisticRegression (TF-IDF)`
Task	Positive (4/5) vs. Negative (1/2)	1 vs. 2 vs. 3 vs. 4 vs. 5-Star
Test F1 (Weighted)	**0.9392**	**0.7347**
Test F1 (Macro)	0.918	0.477
Minority Recall	**92.1%** (for 'Negative')	**10.2%** (for '2-Star')
Overfitting (F1 Gap)	**Low (-2.5%)**	**Low (-10.7%)**
Key Use Case	Real-time customer service alerts.	Exploratory analysis of neutral reviews.

7. Final Remarks and Outlook

7.1. Final Summary of Findings

This project executed a comprehensive comparison of three core NLP paradigms for sentiment analysis. The results were definitive and led to a clear, data-driven strategy.

1. **`Classical (TF-IDF)` Was the Undisputed Winner:** Our initial hypothesis that `Transformer > Embedding > Classical` was proven decisively false. The "simplest" `TF-IDF` approach provided a richer, more direct signal for linear models and dominated all other strategies in both multi-class and binary tasks.

2. **`Embeddings (Word2Vec Mean)` Failed:** The embedding strategy failed. It "blurred" the sentiment signal by averaging vectors, causing the model to learn noise instead of meaning. The SHAP analysis, which showed the model focused on typos (`aplication`) and garbage strings (`hgjlbhjbnjnbb`), was the key diagnostic that proved this method was flawed.

3. **`LogisticRegression` Was the "Smartest" Model:** A `LogisticRegression` model was the recommended winner for *both* tasks.
    - On **TF-IDF data**, it provided the optimal balance of performance and robustness, achieving a top-tier F1-score (0.939) while avoiding the severe overfitting seen in `LinearSVC`.
    - On **Embedding data**, it "won" by *failing correctly*. It produced a stable, non-overfit model that exposed the poor quality of the features, whereas the `LGBMClassifier` overfit the noise.
4. **A Dual-Model Solution is Required:** No single model can serve both business needs.
    - The **Multi-Class Model (F1: 0.735)** is an **analytical tool**. It struggles with the nuanced middle classes but is our only instrument for exploring *why* customers are neutral.
    - The **Binary Model (F1: 0.939)** is a **production tool**. It is highly accurate, robust, and excels at the minority "negative" class (92.1% recall), making it ideal for automated alerts.

7.2. Limitations of this Study

- **The Incomplete Transformer Experiment:** This is the most significant limitation. Our most promising model failed due to an *engineering workflow error* (on-the-fly embedding generation) and not a *model* failure. The true potential of transformers in this problem remains unknown.
- **Flawed Embedding Strategy:** We only tested simple embedding aggregations (mean, TF-IDF). The `Word2Vec (Mean)` strategy was proven to be flawed. More advanced methods (like attention-based aggregation) or domain-specific embeddings were not explored.
- **Single Data Source:** All models were trained on data from one platform. Their ability to generalize to reviews from different sources (e.g., social media, product forums) is not guaranteed.

7.3. Future Work & Recommendations

Based on these findings, the next steps are clear and prioritized:

1. **Priority 1: Fix and Re-run Transformer Experiment.** The top priority is to implement the "Pre-computed" workflow described in Section 6.4. By generating all sentence embeddings once and saving them to disk, we can

run a full Optuna tuning study on a `LGBMClassifier` or `LogisticRegression` using these powerful 384-dimension features. This is the most likely path to a superior multi-class model.

2. **Priority 2: Train Domain-Specific Embeddings.** The `Word2Vec (Mean)` model failed partly because the generic "Google News" embeddings didn't understand our domain. A `Word2Vec` or `FastText` model trained *on our own 1.5M review corpus* would learn relevant vectors for terms like "Petflow," "shipping," and domain-specific slang, which would likely create a much stronger feature set.

3. **Priority 3: Deep-Dive Error Analysis.** A team should manually review the 100-200 worst misclassifications from the multi-class model (e.g., 3-star reviews predicted as 5-star). This qualitative analysis will almost certainly reveal recurring themes (e.g., "The product is great, but the shipping was terrible") that could be engineered into new features (e.g., `has_shipping_complaint`, `has_mixed_sentiment`).

7.4. Final Business Recommendation

This project successfully identified two high-performing, robust models. We recommend a **dual-model deployment strategy** to maximize business value:

1. **For Operations: Deploy the Binary `LogisticRegression (TF-IDF)` Model (F1: 0.939).**
    - **Purpose:** Proactive, real-time intervention.
    - **Use Case:** Integrate this model with the customer review ingestion pipeline. All reviews flagged as "Negative" (with 92.1% recall) should automatically generate a high-priority ticket in the customer service system for immediate follow-up.
2. **For Analytics: Deploy the Multi-Class `LogisticRegression (TF-IDF)` Model (F1: 0.735).**
    - **Purpose:** Exploratory, strategic insight.
    - **Use Case:** This model should be the engine for a business intelligence dashboard. The analytics team can use it (and its SHAP values) to answer deep-dive questions, such as: "What are the top 10 keyword drivers of 3-star reviews this quarter?" or "What is the primary theme separating 2-star from 1-star reviews?"